# Introducción a R

**Maestría en Ciencias Médicas, Odontológicas y de la Salud · UNAM · Semestre 2027-1**

Dra. Yalbi Itzel Balderas-Martínez  Ing. Luis Alberto Meza Cova

12 de agosto de 2026

# Tabla de contenidos


**Presentación** **6**
Cómo usarlo . . . 6
Dónde vamos a trabajar . . . 6
Licencia y cita . . . 6

**1 Encuadre** **7**
1.1 Ciencia de datos . . . 7
1.2 Qué es R . . . 8
1.2.1 Ventajas . . . 8
1.2.2 Desventajas . . . 8
1.3 Qué es RStudio . . . 9
1.4 Dónde vamos a trabajar . . . 9
1.5 Por qué escribir el análisis . . . 9
1.5.1 Un primer ejemplo . . . 9
1.6 De qué trata el curso . . . 10
1.7 Evaluación . . . 10
1.8 Material de apoyo . . . 10
1.9 Sesión de R . . . 10

**2 Primeros pasos en R** **11**
2.1 El caso: dieciocho divisiones a mano . . . 11
2.2 La consola, el script y el entorno . . . 11
2.2.1 La primera línea . . . 12
2.2.2 Cómo se escriben varios comandos . . . 12
2.2.3 No vuelvas a escribir lo mismo . . . 13
2.3 Pedirle ayuda a R . . . 13
2.3.1 Cómo se lee una página de ayuda . . . 13
2.3.2 Ayuda que se ejecuta sola . . . 14
2.4 Paquetes . . . 14
2.4.1 Instalar no es cargar . . . 14
2.5 Calculadora, asignación y nombres . . . 15
2.5.1 Operadores aritméticos . . . 15
2.5.2 Asignar: guardar para después . . . 16
2.5.3 Cómo nombrar una variable . . . 16
2.6 El caso, resuelto . . . 17
2.6.1 Tasa de incidencia . . . 17
2.6.2 Prevalencia y letalidad . . . 18
2.7 Demostraciones . . . 19
2.8 Lo que te llevas . . . 19
2.9 Antes de la próxima sesión . . . 20

**3 Tipos y estructuras de datos** **21**
3.1 El caso: por qué `"42" + 10` no es 52 . . . 21
3.2 Los tres tipos básicos . . . 22
3.2.1 Qué contesta `class()` . . . 22
3.2.2 Los tres errores clásicos . . . 23
3.2.3 Un paréntesis: eso que usaste es una función . . . 23
3.3 Vectores . . . 23
3.3.1 Qué pasa si mezclas tipos . . . 23
3.3.2 Otras formas de crear vectores . . . 24
3.3.3 Nombres . . . 24
3.3.4 Cuatro formas de seleccionar . . . 25
3.3.5 Operaciones vectorizadas . . . 25
3.3.6 Cuando los vectores no miden lo mismo: el reciclado . . . 26
3.4 Matrices . . . 28
3.4.1 R llena las matrices por columnas . . . 28
3.4.2 Nombres de filas y de columnas . . . 29
3.4.3 Añadir filas y columnas . . . 29
3.4.4 Mirar sin imprimir . . . 30
3.4.5 Selección: `matriz[fila, columna]` . . . 30
3.4.6 Selección por condición . . . 31
3.4.7 La coma no es un adorno . . . 31
3.4.8 Una sola fila deja de ser matriz . . . 31
3.4.9 Un `NA` hace una fila fantasma . . . 32
3.4.10 Combinar condiciones . . . 32
3.4.11 Resúmenes por dimensión . . . 34
3.5 Marcos de datos (data frames) . . . 35
3.5.1 Inspeccionarlo . . . 36
3.5.2 Seleccionar . . . 36
3.5.3 Filtrar por condición . . . 36
3.5.4 Añadir una columna . . . 37
3.5.5 Un data frame de verdad: `cars` . . . 37
3.6 Factores . . . 38
3.6.1 Dónde está el factor entre los vectores . . . 39
3.6.2 Cuando el texto es en realidad una categoría . . . 40
3.6.3 Los datos y el catálogo . . . 40
3.6.4 Un conjunto cerrado . . . 41
3.6.5 Contar categorías . . . 41
3.6.6 Lo que se ve y lo que se guarda . . . 41
3.6.7 Declarar el orden: `levels` y `relevel()` . . . 42
3.6.8 Categorías que tienen orden . . . 43
3.6.9 `levels` ordena, `ordered` compara . . . 43
3.6.10 De números a categorías: `cut()` . . . 44
3.6.11 La trampa de `as.numeric()` . . . 45
3.6.12 Para qué sirven de verdad . . . 46
3.6.13 Datos reales: categorías guardadas como números . . . 46
3.6.14 El mismo grupo, dos modelos . . . 47
3.6.15 La categoría de referencia . . . 48

3.7 Listas . . . 50
3.7.1 Cuatro cosas que no caben juntas . . . 51
3.7.2 Mirar una lista . . . 51
3.7.3 Sacar cosas de una lista: la analogía del tren . . . 52
3.7.4 Un corchete de menos . . . 53
3.7.5 `$`: el atajo por nombre . . . 53
3.7.6 Por posición, y con el nombre en una variable . . . 53
3.7.7 Estructuras dentro de la lista . . . 54
3.7.8 Añadir, cambiar y quitar elementos . . . 54
3.7.9 Cuando cada caso mide distinto . . . 54
3.7.10 `unlist()`: aplanar, y la coerción que vuelve . . . 55
3.7.11 El data frame es una lista . . . 56
3.7.12 Los modelos son listas . . . 56
3.8 La escalera, de un vistazo . . . 58
3.9 Antes de la próxima sesión . . . 59

**4 Visualización con ggplot2** **60**
4.1 El problema del catálogo de gráficos . . . 60
4.1.1 La otra respuesta: una gramática . . . 60
4.1.2 R base contra ggplot2 . . . 61
4.1.3 Los ocho componentes . . . 61
4.2 Datos ordenados: la condición de entrada . . . 62
4.2.1 Por qué le importa a ggplot2 . . . 62
4.3 Los datos: `mpg` . . . 62
4.4 La gráfica mínima . . . 63
4.4.1 El `+` no es casualidad . . . 63
4.4.2 El mapeo por defecto . . . 63
4.5 Estéticas . . . 64
4.5.1 Mapear no es fijar . . . 64
4.6 Geometrías . . . 65
4.7 Facetas . . . 65
4.7.1 Color o faceta: no es lo mismo . . . 66
4.7.2 Dos variables a la vez . . . 66
4.8 Escalas . . . 66
4.9 Etiquetas . . . 67
4.10 Tema . . . 68
4.11 La gráfica completa . . . 68
4.12 Lo que te llevas . . . 69
4.13 Lo que te llevas del curso . . . 69
4.14 A dónde seguir . . . 69

**I Anexos** **70**

**Anexo · Posit Cloud** **71**
Crear tu cuenta . . . 71
Qué vas a ver . . . 71
Por qué en la nube y no en tu computadora . . . 71

Límite del plan gratuito . . . 72
Si prefieres instalarlo en tu computadora . . . 72

**Anexo · Glosario** **73**

**5 Anexo · Errores frecuentes** **74**
5.1 Errores que R sí reporta . . . 74
5.1.1 `could not find function "..."` . . . 74
5.1.2 `objeto 'x' no encontrado` . . . 74
5.1.3 `argumento no-numérico para operador binario` . . . 74
5.1.4 `subíndice fuera de los límites` . . . 75
5.1.5 `no se puede abrir la conexión` / `cannot open file` . . . 75
5.2 Avisos que no son errores, y son peores . . . 75
5.2.1 `NAs introducidos por coerción` . . . 75
5.2.2 Coerción silenciosa en un vector . . . 76
5.2.3 `longitud del objeto mayor no es múltiplo de la longitud del menor` 76
5.2.4 Un factor al que le asignas un nivel que no existe . . . 76
5.3 Resultados que parecen errores y no lo son . . . 76
5.3.1 `named numeric(0)`, `character(0)`, `integer(0)` . . . 76
5.3.2 `NULL` . . . 76
5.3.3 `[1]` al principio de cada línea . . . 76
5.3.4 Una gráfica que no aparece . . . 77
5.4 Confusiones de concepto . . . 77
5.5 Cómo pedir ayuda de forma que se pueda contestar . . . 77

**Referencias** **78**

# Presentación

Este es el manual del curso **Introducción a R** del Programa de Maestría y Doctorado en Ciencias Médicas, Odontológicas y de la Salud de la UNAM, semestre 2027-1.

Las diapositivas son para la clase; este manual es para después: tiene la explicación completa, los ejemplos con su salida y las referencias.

## Cómo usarlo

Cada capítulo corresponde a una sesión. Puedes leerlo en orden o entrar directo al tema que necesitas desde el buscador.

Los bloques de código se pueden copiar con el botón de la esquina y pegar directamente en RStudio.

## Dónde vamos a trabajar

En este curso no instalas nada: usamos Posit Cloud, que es RStudio dentro del navegador. Ver el anexo de Posit Cloud para crear tu cuenta.

## Licencia y cita

El material se puede compartir y adaptar con atribución y sin uso comercial (CC BY-NC 4.0). Si lo reutilizas, cita como se indica en el archivo `CITATION.cff` del repositorio.

# 1 Encuadre

Este capítulo es la versión escrita del deck `slides/00-encuadre.qmd`. Su contenido proviene de la clase 0 de la edición anterior del curso, reorganizado para leerse fuera del aula.

## 1.1. Ciencia de datos

La ciencia de datos es un campo interdisciplinario que usa métodos, procesos, algoritmos y sistemas científicos para extraer conocimiento de datos, estén organizados en una tabla o no.

La forma más común de describirla es como la intersección de tres círculos: matemáticas y estadística, cómputo, y **conocimiento del dominio**.

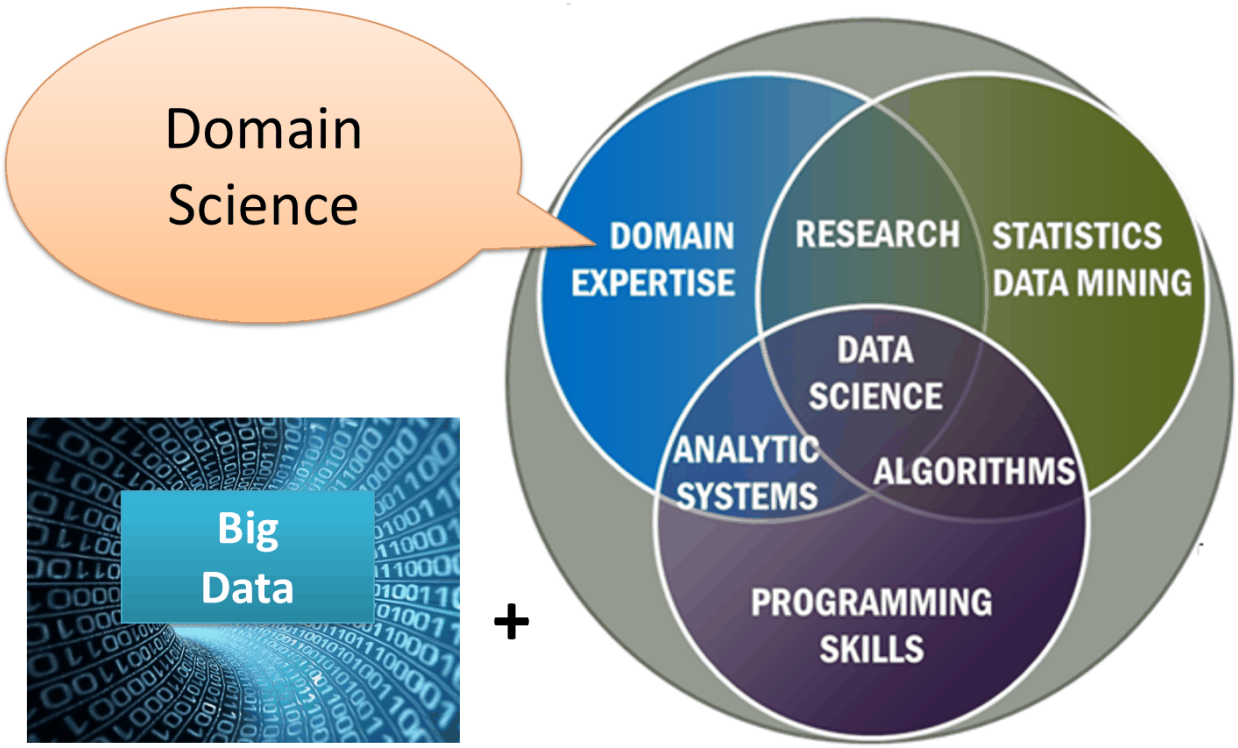



Figura 1.1: Ciencia de datos según el NIST Big Data Working Group

Ese tercer círculo es el que importa aquí. En biología y salud, el conocimiento del dominio es el que tú ya traes: es la parte que no se aprende en un curso de programación.

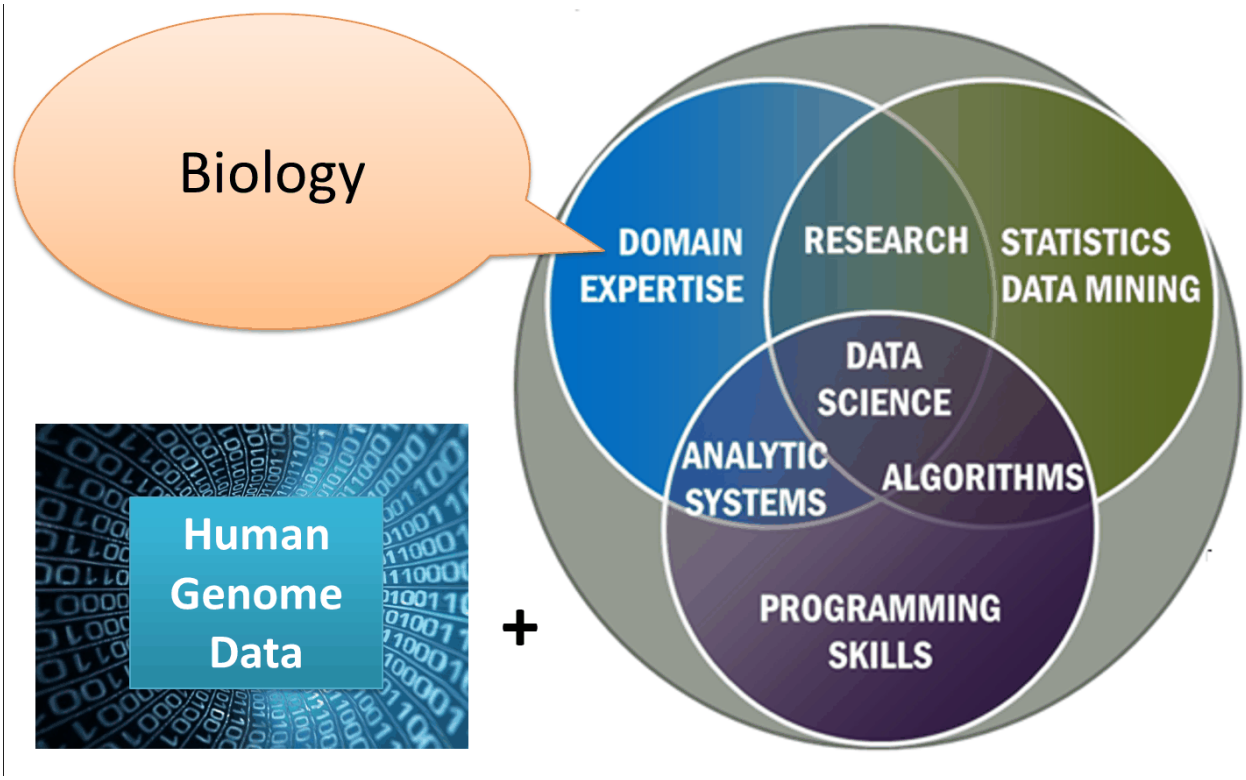



Figura 1.2: El mismo diagrama, aplicado a biología

## 1.2. Qué es R

R es un ambiente de software libre para cómputo estadístico y gráficos. Compila y corre en Windows, macOS y Linux.

Lo desarrollaron personas expertas en estadística a partir del lenguaje S, creado por John Chambers y colegas en los Laboratorios Bell (hoy Lucent Technologies).

### 1.2.1. Ventajas

- Es libre: no depende de una licencia que se venza ni de un presupuesto.
- Las figuras y gráficos que produce son de calidad de publicación.
- Se extiende instalando paquetes.
- Fue hecho por y para gente que analiza datos, no adaptado desde otro fin.

### 1.2.2. Desventajas

- Requiere tiempo de entrenamiento: para personalizar un resultado hay que conocer el lenguaje.
- Algunos paquetes están mal documentados, abandonados, o no son la mejor herramienta para el problema.
- Ciertas tareas se resuelven más fácil en otro lenguaje.

## 1.3. Qué es RStudio

Un ambiente de desarrollo integrado (IDE): consola, editor, herramientas gráficas y manejo del área de trabajo en una sola ventana.

R es el motor; RStudio es el tablero desde el que lo manejas. Puedes usar R sin RStudio, pero es más incómodo.

## 1.4. Dónde vamos a trabajar

En este curso **no instalas nada**. Usamos Posit Cloud: el mismo RStudio, corriendo en un servidor en vez de en tu computadora. Se abre en el navegador con una cuenta gratuita.

La razón es práctica. Instalar R y RStudio en cada computadora convierte la primera sesión en soporte técnico —versiones distintas, permisos de administrador, antivirus— y se va la clase sin haber escrito una línea de R. En la nube todas trabajamos sobre el mismo R, con los mismos paquetes, y un problema se resuelve una sola vez.

No es una versión reducida: lo que aprendas aquí funciona igual el día que instales R en tu máquina.

Cómo crear la cuenta y qué esperar del plan gratuito: ver el anexo de Posit Cloud.

## 1.5. Por qué escribir el análisis

En una hoja de cálculo, las operaciones se hacen a mano y no queda registro de ellas. Seis meses después nadie —ni tú— puede decir con certeza qué celdas se tocaron.

En R, cada paso se escribe como una instrucción, y el archivo de instrucciones **es** el registro del análisis. Si un revisor pregunta cómo obtuviste una cifra, la respuesta es ese archivo: se vuelve a correr y produce el mismo resultado.

Esa propiedad se llama **reproducibilidad** y es la razón de fondo de este curso.

### 1.5.1. Un primer ejemplo

```
edades <- c(34, 41, 28, 55, 47)
mean(edades)
```

```
[1] 41
```

La primera línea guarda cinco edades en un objeto llamado `edades`. La segunda pide su promedio.

El nombre va en plural a propósito: guarda cinco valores, no uno. En la sesión 01 se usa `edad` para un solo número y `edades` para el vector, y conviene que diga lo mismo desde aquí.

## 1.6. De qué trata el curso

Un curso introductorio para aprender a programar en R y adquirir nociones de cómo analizar tus propios datos. No se requieren conocimientos previos de programación.

## 1.7. Evaluación

| Componente | Peso |
|---|---|
| Tareas en DataCamp | 20 % |
| Primer examen | 25 % |
| Segundo examen | 25 % |
| Proyecto final en R | 30 % |

Asistencia mínima: **80 %**.

> **Nota**
>
> Criterios heredados de la edición anterior del curso. Se confirman para el semestre 2027-1 en `docs/programa-operativo/`.

## 1.8. Material de apoyo

Además del material de este repositorio, el curso ha usado DataCamp a través del programa *DataCamp for the Classroom*, que da acceso completo a la plataforma mientras el curso esté inscrito.

Lecturas de apoyo sobre qué es la ciencia de datos y cómo aprender R:

- https://www.datacamp.com/es/blog/what-is-data-science-the-definitive-guide
- https://www.datacamp.com/blog/learn-r

## 1.9. Sesión de R

Al cerrar cada análisis conviene registrar con qué versiones se corrió. Es lo que permite explicar, meses después, por qué un resultado cambió:

```
sessioninfo::session_info()
```

# 2 Primeros pasos en R

**Tema 01** · unidad teórico-práctica

> **Objetivo**
>
> Trabajar en R con soltura suficiente para no depender de que alguien dicte el comando: pedirle ayuda al propio R, instalar y cargar un paquete, guardar resultados en variables con nombres que sobrevivan a la semana siguiente, y escribir un cálculo que se pueda volver a correr cuando cambien los datos.

## 2.1. El caso: dieciocho divisiones a mano

Tienes los datos epidemiológicos de seis estados y te piden tres indicadores de cada uno: tasa de incidencia, prevalencia y letalidad. Son tres divisiones por estado. Dieciocho en total.

Con una calculadora se hacen en diez o quince minutos, si no te equivocas. El problema no es ese. El problema llega el mes siguiente, cuando alguien manda la corrección de las cifras de dos estados y hay que rehacerlo todo. Y otra vez en abril.

Ese es el trabajo que R elimina, y por eso el curso abre aquí y no con `1 + 1`.

> **Lo que hace distinto a R**
>
> Una calculadora te da **el resultado**. R te deja **el procedimiento escrito**.
> Un procedimiento escrito se vuelve a correr, se revisa, se corrige y se le manda a alguien más.
> Un resultado, no.

Es el argumento del curso entero y conviene tenerlo presente en las tres sesiones: no venimos a aprender una herramienta de cálculo, venimos a dejar constancia de cómo se calculó.

## 2.2. La consola, el script y el entorno

Al abrir RStudio aparecen cuatro paneles. Hoy solo importan dos.

| Panel | Qué es | Para qué |
|---|---|---|
| **Consola** | donde R contesta | probar |
| **Editor** | donde escribes el archivo `.R` | **guardar** |
| Entorno | lo que R tiene en memoria | ver qué existe |
| Archivos / Gráficas / Ayuda | el resto | mirar la ayuda y las figuras |

R y RStudio no son lo mismo. **R** es el lenguaje y el intérprete; **RStudio** es la ventana desde la que se le habla. Fuera de este curso se instalan por separado y en ese orden; aquí los dos vienen ya montados en Posit Cloud (ver el anexo de Posit Cloud).

La diferencia entre los dos primeros paneles es la que ordena todo lo demás: lo que escribes en la consola se ejecuta **y se pierde**; lo que escribes en el script se guarda y se puede volver a correr entero.

> **!** La regla de la primera sesión
>
> Se prueba en la consola, se guarda en el script. Todo lo que quieras volver a usar la semana que viene tiene que acabar en un archivo `.R`.

El error de la primera semana casi nunca es de sintaxis: es haber hecho todo el trabajo en la consola y cerrar RStudio. El atajo que evita eso es `Ctrl+Enter` (`Cmd+Enter` en Mac), que manda al intérprete la línea del script donde está el cursor.

### 2.2.1. La primera línea

```
3 + 4
#> [1] 7
```

El `[1]` no es parte del resultado: es **la posición del primer elemento** de lo que R está imprimiendo. Con un solo número parece ruido. Va a tener sentido completo en el tema de tipos y estructuras de datos, cuando el resultado tenga cien elementos y R los reparta en varias líneas: cada línea empieza indicando por qué elemento va.

### 2.2.2. Cómo se escriben varios comandos

```
# Con salto de línea: lo normal
casos <- 125
poblacion <- 5000

# Con punto y coma: dos en la misma línea
casos <- 125; poblacion <- 5000

# Entre llaves: un grupo que se ejecuta junto
{ casos <- 125; poblacion <- 5000 }
```

El `#` comenta hasta el final de la línea. El punto y coma existe y se lee peor; sirve para dos asignaciones cortas y emparentadas, no para juntar tres cosas distintas.

Hay un límite que parece curiosidad y explica un error real: **un comando introducido en la consola no puede pasar de 4 095 bytes**. Si pegas un bloque enorme copiado de internet, se corta. En un script no pasa.

### 2.2.3. No vuelvas a escribir lo mismo

- ↑ y ↓ en la consola recorren lo que ya escribiste.
- **Ctrl**+↑ (`Cmd`+↑) abre la lista completa del historial.
- **Tab** completa nombres de funciones, de variables y de rutas de archivo.

De los tres, el más rentable es **Tab**. Completa y, de paso, te dice si el objeto existe: si no aparece en la lista, o lo escribiste mal o todavía no lo has creado.

## 2.3. Pedirle ayuda a R

R trae su documentación dentro. No hace falta salir a buscar en internet para saber qué argumentos acepta una función.

| Escribes | Cuándo | Qué te da |
|---|---|---|
| `?read.table` | **sabes** el nombre exacto | la página de ayuda de esa función |
| `??solve` | no lo sabes exacto | busca en todo lo instalado |
| `help.search("data input")` | buscas por tema | lo mismo que `??`, admitiendo frases |
| `apropos("lm")` | recuerdas un trozo del nombre | los nombres que **contienen** ese trozo |
| `find("lowess")` | sabes el nombre, no el paquete | en qué paquete vive |

La regla corta: **una interrogación si sabes el nombre, dos si no**.

`help.start()` abre la documentación completa en el navegador. Es útil el primer día y casi nunca después.

### 2.3.1. Cómo se lee una página de ayuda

Todas tienen las mismas secciones, en el mismo orden:

| Sección | Qué contesta |
|---|---|
| **Description** | qué hace |
| **Usage** | cómo se llama, con sus argumentos y valores por defecto |
| **Arguments** | qué significa cada argumento |
| **Value** | **qué devuelve** |
| **Examples** | código que puedes copiar y correr |

Casi nadie las lee de arriba abajo. Lo práctico es ir a **Usage**, saltar a **Examples**, y volver a **Arguments** cuando algo no cuadra.

Aprender a leer la ayuda vale más que memorizar veinte funciones: las veinte funciones se buscan, y leer la ayuda no se aprende solo.

### 2.3.2. Ayuda que se ejecuta sola

```
example(sum)
#> sum> sum(1:5)
#> [1] 15
#> sum> sum(1:5, NA)
#> [1] NA
#> sum> sum(1:5, NA, na.rm = TRUE)
#> [1] 15
```

`example()` corre los ejemplos de la página de ayuda dentro de tu sesión. Este en concreto es un regalo: en tres líneas enseña la función, enseña que un valor faltante (`NA`) contamina el resultado, y enseña el argumento que lo excluye.

De momento basta con esta idea de `NA`: hay un hueco en los datos, y R prefiere avisarte a inventarse un número. Vuelve en el tema de tipos y estructuras de datos.

Ejercicio · encuentra la función

1. ¿En qué paquete vive `lowess`?
2. Busca funciones cuyo nombre contenga `mean`. ¿Cuántas salen?
3. Abre la ayuda de `round`. ¿Qué hace el argumento `digits`?
4. Corre los ejemplos de `round` sin copiarlos a mano.

Respuestas: `find("lowess")` · `apropos("mean")` · `?round` · `example(round)`.
La segunda no tiene un número correcto: depende de los paquetes que tengas instalados. Esa es la lección.

## 2.4. Paquetes

Cuando instalas R ya vienen paquetes de base: `base`, `stats`, `utils`, `graphics` y algunos más. `sum()`, `c()` y `mean()` viven ahí. El índice completo del paquete `base` está en la documentación oficial (R Core Team, 2024).

Lo demás lo escribe gente ajena al equipo que desarrolla R y se publica en **CRAN**, el repositorio oficial. Que R sea extensible por terceros es la razón de que exista `ggplot2`, que es todo el tema 04.

### 2.4.1. Instalar no es cargar

```
install.packages("ggplot2")   # UNA vez, en la vida de tu computadora
library(ggplot2)              # CADA vez que abres R
```

`install.packages()` **descarga** el paquete a tu disco. `library()` lo **pone disponible** en la sesión que tienes abierta. Son dos cosas distintas y hacen falta las dos.

> ⚠ El error de la primera semana
>
> `Error in ggplot(...) : could not find function "ggplot"`
>
> El paquete está instalado. Lo que falta es `library(ggplot2)` **en esta sesión**.

La analogía que funciona: instalar es comprar el libro; cargar es sacarlo del estante y abrirlo. Comprarlo dos veces no sirve de nada, y tenerlo en el estante tampoco si no lo abres.

Por eso, en los scripts que se comparten, la línea de `install.packages()` va **comentada**: si no, se reinstala el paquete cada vez que alguien corre el archivo.

```
# install.packages("ggplot2")   # comentado a propósito
library(ggplot2)
d <- data.frame(x = rnorm(100))
ggplot(data = d) + geom_histogram(aes(x = x), bins = 30)
```

Esas cuatro líneas generan cien números al azar de una distribución normal, los meten en una tabla y dibujan su histograma. Hoy no hay que entenderlas: hay que ver que **funcionan**. El resto del curso consiste en entenderlas —la tabla en el tema de tipos y estructuras de datos, y la gráfica en el de ggplot2—.

## 2.5. Calculadora, asignación y nombres

### 2.5.1. Operadores aritméticos

```
3 + 4      #> 7
3 - 4      #> -1
3 * 4      #> 12
3 / 4      #> 0.75
3 ^ 4      #> 81
7 %/% 2    #> 3     cociente entero
7 %% 2     #> 1     resto
```

Los dos últimos parecen exóticos y no lo son: `%%` es como se pregunta en R si un número es par (`x %% 2 == 0`).

> 💡 Ejercicio · piensa un número
>
> Piensa un número cualquiera. Multiplícalo por 5, súmale 1, multiplica el resultado por 2, réstale 12, divide entre 10 y réstale el número inicial.
> A todo el mundo le da **−1**.
> No es magia, es álgebra:

$$\frac{(5x+1)\cdot 2-12}{10}-x=\frac{10x-10}{10}-x=x-1-x=-1$$

El número que pensaste se cancela. Escribirlo en R con una variable — `x <- 7; ((x * 5 + 1) * 2 - 12) / 10 - x` — y probar con varios valores es el argumento más corto a favor de escribir el procedimiento y no solo el resultado.

### 2.5.2. Asignar: guardar para después

Asignar es guardar un valor en la memoria del ordenador e identificarlo con un nombre, para acceder a él rápidamente después.

```
x <- 4    # el operador de R
x = 5     # también funciona
```

Los dos operadores asignan. **En R se usa `<-`**, y no es capricho: el signo `=` ya significa otra cosa dentro de una llamada a función.

```
seq(from = 0, to = 1, by = 0.25)
```

Ahí `from = 0` **no crea una variable**: pasa un argumento con nombre. El operador `<-` significa siempre lo mismo, en cualquier posición, así que quien lea tu código sabrá sin pensar cuál de las dos cosas está pasando.

El atajo de RStudio para escribirlo es `Alt+-` (`Option+-` en Mac). Conviene aprenderlo el primer día: si hay que teclear los dos caracteres a mano, se acaba usando `=`.

Y hay algo que sorprende al principio: **asignar no imprime nada**. Para ver el contenido de un objeto se escribe su nombre.

```
tasa_incidencia <- 25   # no imprime nada
tasa_incidencia         # ahora sí
#> [1] 25
```

Quien venga de otro lenguaje va a buscar un `print()`. Existe, y aquí no hace falta.

### 2.5.3. Cómo nombrar una variable

R distingue mayúsculas de minúsculas: `casos`, `Casos` y `CASOS` son tres variables distintas. Y `aun` no es `aún`.

| Regla | Mal | Bien |
|---|---|---|
| No empezar con dígito | `1casos` | `casos1` |
| El `_` no va al principio | `_casos` | `casos_marzo` |
| Tras un `.` inicial debe ir letra, no dígito | `.1casos` | `.casos` |

| Regla | Mal | Bien |
|---|---|---|
| Máximo 256 bytes | — | nombres cortos |
| Portable: solo `A-Z a-z 0-9 _ .` | `población` | `poblacion` |

Los acentos y las eñes funcionan en muchos sistemas y fallan en otros. Lo peor es que fallan al abrir el archivo en otra computadora, es decir, lejos de donde se cometió el error. **Regla de la casa: sin acentos ni eñes en los nombres de objeto**; en los comentarios, los que hagan falta.

Más allá de lo que R acepta, está lo que se entiende:

```
# Se entiende hoy y dentro de un mes
casos_confirmados <- 125
poblacion_riesgo  <- 5000
tasa_incidencia   <- (casos_confirmados / poblacion_riesgo) * 1000

# Se entiende hoy
a <- 125; b <- 5000
c <- (a / b) * 1000
```

El segundo bloque no está mal: está mudo. Y tiene un problema añadido —`c` pisa el nombre de la función `c()`, que es la que más se usa al trabajar con vectores—. Pisar nombres de funciones básicas produce errores que aparecen mucho después y son difíciles de rastrear.

La convención de este curso, que es la del material entero: minúsculas y guion bajo (`snake_case`), sin acentos, con un sustantivo que diga qué guarda. Nada de `camelCase`.

## 2.6. El caso, resuelto

### 2.6.1. Tasa de incidencia

Mide la aparición de **casos nuevos** de una enfermedad en una población en riesgo, durante un periodo de tiempo.

$$TI = \frac{C}{N} \times k$$

donde $C$ son los casos nuevos en el periodo, $N$ la población en riesgo y $k$ una constante de escala (1 000; 100 000…). La constante solo cambia la unidad en la que se lee el resultado —«por cada mil habitantes», «por cada cien mil»—; no cambia el fenómeno.

```
casos_confirmados <- 125     # casos nuevos
poblacion_riesgo  <- 5000    # población en riesgo

# Tasa de incidencia por cada 1 000 habitantes
tasa_incidencia <- (casos_confirmados / poblacion_riesgo) * 1000
```

```
tasa_incidencia
#> [1] 25
```

Cada elemento de la fórmula tiene una línea de código con su nombre. Los paréntesis no hacen falta —la división y la multiplicación tienen la misma precedencia y se evalúan de izquierda a derecha— y están para que el código se lea como la fórmula.

### 2.6.2. Prevalencia y letalidad

$$P = \frac{C}{N} \times 100 \qquad L = \frac{D}{C} \times 100$$

La **prevalencia** mide la proporción de personas que tienen la enfermedad en un momento dado: aquí $C$ son **todos** los casos, nuevos y antiguos. La **letalidad** mide la proporción de personas con la enfermedad que mueren a causa de ella: su denominador son **los casos**, no la población.

> ⚠ La confusión clásica
>
> Cambiar el denominador de la letalidad —poner la población en lugar de los casos— cambia el resultado por un factor de veinticinco en el ejemplo de abajo. Y no lanza ningún mensaje: R divide lo que le pongas.

```
casos_confirmados <- 125    # casos nuevos
poblacion_riesgo  <- 5000   # personas en riesgo
muertes           <- 8      # defunciones entre los casos
casos_totales     <- 200    # todos los casos de la enfermedad

tasa_incidencia <- (casos_confirmados / poblacion_riesgo) * 1000
prevalencia     <- (casos_totales     / poblacion_riesgo) * 100
tasa_letalidad  <- (muertes           / casos_totales)    * 100

tasa_incidencia
#> [1] 25
prevalencia
#> [1] 4
tasa_letalidad
#> [1] 4
```

`prevalencia` y `tasa_letalidad` valen **4** las dos y no significan lo mismo: el 4 % de la población tiene la enfermedad, y el 4 % de los enfermos murió. La coincidencia es de este ejemplo, no del mundo, y es un buen recordatorio de que el número solo no dice nada: hace falta saber qué se dividió entre qué. Por eso las variables se llaman como se llaman.

**Ejercicio 1 · cambia los valores**

Asigna otros números de casos confirmados y de población en riesgo, y vuelve a calcular la tasa de incidencia.
**Pregunta:** ¿qué pasa si la población en riesgo crece y los casos confirmados se quedan igual? Fíjate en lo que costó: cambiar **dos líneas** y volver a correr. Eso es exactamente lo que la calculadora no permitía, y es la promesa con la que abrió la sesión.

**Ejercicio 2 · dos poblaciones**

Asigna valores para dos poblaciones distintas —`poblacion_riesgo1` y `poblacion_riesgo2`—, calcula la tasa de incidencia en las dos y compáralas.
**Pregunta:** ¿cuál de las dos poblaciones tiene mayor riesgo relativo?

Con dos poblaciones ya hay cuatro variables. Con los seis estados del caso serían doce, y el código dejaría de caber en la pantalla. Esa incomodidad tiene nombre —se llama **vector**— y ocupa una línea. Es el tema de tipos y estructuras de datos.

## 2.7. Demostraciones

R trae demostraciones que enseñan de lo que es capaz sin que haya que escribir nada:

```
demo(persp)      # superficies en perspectiva
demo(graphics)   # el repertorio de gráficos base
demo(Hershey)    # tipografías vectoriales
demo(plotmath)   # fórmulas matemáticas dentro de una gráfica
```

No entran en el examen de nada. Sirven para ver el techo de la herramienta el primer día, que es cuando más falta hace.

## 2.8. Lo que te llevas

- Se prueba en la consola, **se guarda en el script**.
- Una interrogación si sabes el nombre; dos si no.
- `install.packages()` una vez; `library()` cada sesión.
- `<-` para asignar, siempre.
- Nombres en minúsculas, con `_`, sin acentos, y que digan qué guardan.

Y si algo se te olvida, que no sea esto: **el valor de R no es que calcule, es que deja escrito cómo calculó.**

## 2.9. Antes de la próxima sesión

1. Entra a tu proyecto de Posit Cloud y comprueba que abre sin errores (ver el anexo de Posit Cloud). No tienes que instalar nada.
2. Corre `install.packages("ggplot2")` una vez en tu proyecto: se usa en la tema 04.
3. Termina los ejercicios 1 y 2 en un archivo `.R` guardado.

Si algo no instala, escríbeme antes del miércoles. Resolver una instalación rota en vivo cuesta quince minutos de clase; por correo, cinco.

# 3 Tipos y estructuras de datos

**Tema 02** · unidad teórico-práctica

> **Objetivo**
>
> Elegir la estructura de datos que corresponde al problema —vector, matriz, factor, data frame o lista— y justificar la elección; y seleccionar subconjuntos de datos por posición, por nombre y por condición lógica sin escribir un solo ciclo.

Este capítulo es una escalera, no una lista de temas. Cada estructura aparece porque **la anterior se queda corta** con el problema que sigue. Si se lee como cinco cosas que memorizar, se pierde lo único que importa: reconocer qué te está pidiendo el problema que tienes delante.

## 3.1. El caso: por qué `"42" + 10` no es 52

```
y   <- 10
num <- "42"
y + num
#> Error in y + num : argumento no-numérico para operador binario
```

Para ti los dos «son 42 y 10». Para R, uno es un **número** y el otro es **texto**. El mensaje de error es de los buenos: dice exactamente qué pasó.

Es, además, el error que más aparece en la vida real, cuando se abre un CSV en el que una columna traía un espacio de más, un guion o una coma decimal.

```
class(num)
#> [1] "character"
class(y)
#> [1] "numeric"

num <- as.numeric(num)   # convertir el texto a número
y + num
#> [1] 52
```

`class()` es la función que más se usa hoy, y `as.numeric()` inaugura una familia entera: `as.character()`, `as.logical()`, `as.factor()`.

> **Advertencia**
>
> `as.numeric("hola")` devuelve `NA` **con una advertencia**, no con un error. Convertir a ciegas es como se pierden datos en silencio: conviene mirar cuántos `NA` aparecieron después de convertir.

## 3.2. Los tres tipos básicos

```
a  <- 5        # numérico
b  <- 3.14     # numérico
c2 <- 2e3      # numérico: 2000

x <- 5; y <- 10
x < y          # lógico: TRUE
x == y         # lógico: FALSE

nombre <- "Luis"     # carácter
curso  <- 'IntroR'   # las comillas simples son equivalentes
```

Tres apuntes sobre esto:

- `a <- 5` da `"numeric"`, no `"integer"`. R guarda los números como `double` por defecto; para un entero de verdad hace falta escribir `5L`.
- Las comillas dobles y las simples son equivalentes. Elige unas y no las mezcles dentro de un archivo.
- El nombre `c2` está a propósito: `c` pisaría la función `c()`, que es la que se usa en todo el resto del capítulo (Sección 2.5.3).

### 3.2.1. Qué contesta `class()`

```
class(42)        #> "numeric"
class("42")      #> "character"
class(TRUE)      #> "logical"
class("TRUE")    #> "character"
class(42 > 10)   #> "logical"
class(NA)        #> "logical"
```

Con comillas es **texto**, siempre: da igual que dentro haya un número o la palabra `TRUE`. Los pares que se parecen sin ser lo mismo son el primero con el segundo y el tercero con el cuarto. El quinto muestra que una comparación **devuelve** un lógico, en lugar de escribirlo uno. Y `class(NA)` da `"logical"`: por ahora basta con saber que `NA` también tiene tipo.

Esa regla —con comillas es texto— resuelve la mayoría de los problemas de tipos que aparecen con datos reales.

### 3.2.2. Los tres errores clásicos

```
"42" + 10          # 1. operar entre tipos incompatibles -> Error

logico <- TRUE     # 2. "logical"
texto  <- "TRUE"   #    "character" - no es lo mismo

nombre <- Luis     # 3. sin comillas -> Error: objeto 'Luis' no encontrado
```

El tercero es el traicionero. Sin comillas, R no interpreta `Luis` como texto: busca **un objeto** llamado `Luis`. Si no existe, avisa. Si existe, obedece y te da su contenido, que casi nunca es lo que querías. Un error se convierte así en un bug silencioso.

### 3.2.3. Un paréntesis: eso que usaste es una función

`class()` y `as.numeric()` son **funciones**: bloques de código con nombre que cumplen una tarea y se reutilizan tantas veces como haga falta, en vez de repetir líneas. Se reconocen por los paréntesis que siguen al nombre, y `?nombre` dice qué argumentos aceptan.

Escribir funciones propias es materia de otro curso. Pero conviene nombrar la idea aquí, porque de este punto en adelante todo lo que se hace es llamar funciones.

## 3.3. Vectores

Un **vector** es una colección **ordenada** de elementos **del mismo tipo**.

```
v_num <- c(1, 2, 3, 4.5)
v_int <- c(1L, 2L, 3L)        # la L fuerza un entero de verdad
v_log <- c(TRUE, FALSE, TRUE)
v_chr <- c("a", "b", "c")
```

`c()` viene de *combine*. Y «del mismo tipo» no es un detalle de implementación: es la definición.

### 3.3.1. Qué pasa si mezclas tipos

```
c(1, 2, "tres")
#> [1] "1"    "2"    "tres"

c(TRUE, FALSE, 3)
#> [1] 1 0 3
```

R **no da error**: convierte todo al tipo que lo admite todo. Se llama **coerción** y sigue este orden: `lógico → numérico → carácter`.

> ⚠ Advertencia
>
> Tus números se volvieron texto **sin avisar**. Tres líneas más abajo, un `sum()` falla, y el error aparece muy lejos de donde se originó.
> Regla práctica: si un `sum()` o una comparación fallan sin motivo aparente, `class()` al vector antes que nada.

Esta es la razón profunda de que a R le importen los tipos, y la que explica por qué el capítulo empieza por ahí y no por las estructuras.

### 3.3.2. Otras formas de crear vectores

```
1:6
#> [1] 1 2 3 4 5 6

seq(from = 0, to = 1, by = 0.25)
#> [1] 0.00 0.25 0.50 0.75 1.00

rep(0, times = 5)
#> [1] 0 0 0 0 0
```

`1:6` es el atajo de cada día. `seq()` hace falta cuando el paso no es 1, y `rep()` cuando hay que inicializar algo.

### 3.3.3. Nombres

Imagina una lista de casos confirmados en distintos estados. No basta con tener los números: hace falta saber a qué estado corresponde cada uno.

```
casos_marzo <- c(1200, 850, 430, 670, 1500, 920)
names(casos_marzo) <- c("CDMX", "Jalisco", "Yucatan",
                        "Puebla", "NuevoLeon", "Chiapas")
casos_marzo
#>      CDMX   Jalisco   Yucatan    Puebla NuevoLeon   Chiapas
#>      1200       850       430       670      1500       920
```

> ❗ Importante
>
> `names()` **no junta dos vectores**. Le pega una etiqueta a cada posición del vector que ya existe: los datos siguen siendo exactamente los mismos.
> `names` es un **atributo** del vector, no una segunda columna. Sirve para identificar cada posición con un texto, y para leer sin contar posiciones: `casos_marzo["Puebla"]` devuelve `670`.

Aquí se cierra lo que quedó pendiente en el Capítulo 2: aquellas doce variables sueltas para seis estados son ahora dos vectores.

### 3.3.4. Cuatro formas de seleccionar

```
defunciones_marzo <- c(50, 40, 12, 25, 65, 30)
names(defunciones_marzo) <- names(casos_marzo)

defunciones_marzo[1]                          #> 50              por posición
defunciones_marzo[c(2, 5)]                    #> 40 65           varias
defunciones_marzo[-3]                         #> 50 40 25 65 30  quita la tercera
defunciones_marzo[defunciones_marzo > 30]     #> 50 40 65        por condición
```

Dos cosas sorprenden aquí:

- **Los índices empiezan en 1**, no en 0. Quien venga de Python lo va a tropezar durante un par de sesiones.
- **El índice negativo excluye**, no cuenta desde el final. `[-3]` significa «todos menos el tercero».

Y la cuarta forma es la importante, porque se repite en todas las estructuras que quedan.

```
defunciones_marzo > 30
#>      CDMX   Jalisco   Yucatan    Puebla NuevoLeon   Chiapas
#>      TRUE      TRUE     FALSE     FALSE      TRUE     FALSE
```

La condición **no devuelve los datos**: devuelve un vector de `TRUE` y `FALSE` del mismo largo. Ese vector es lo que va dentro de los corchetes, y R se queda con las posiciones que valen `TRUE`.

Entender ese paso intermedio es lo que hace que las matrices y los data frames sean fáciles en lugar de mágicos. Vale la pena correrlo en dos pasos: primero la condición sola, después dentro de los corchetes.

### 3.3.5. Operaciones vectorizadas

```
casos_enero <- c(3000, 1500, 1200, 1800, 2500, 2100)
names(casos_enero) <- names(casos_marzo)

casos_enero + casos_marzo
#>      CDMX   Jalisco   Yucatan    Puebla NuevoLeon   Chiapas
#>      4200      2350      1630      2470      4000      3020
```

R aplica la operación **elemento por elemento**, posición por posición, sin que escribas un ciclo. Se llama **vectorización** y es la razón de que R resulte cómodo para trabajar con datos.

Quien haya programado en otro lenguaje espera un `for`. En R casi nunca hace falta, y cuando aparece suele ser señal de que existe una forma más corta.

Junto a esa familia hay otra, la de **agregación**, que resume el vector entero en un solo número:

```r
sum(casos_enero)    #> 12100
sum(casos_marzo)    #> 5570
mean(casos_enero)   #> 2016.667
min(casos_marzo)    #> 430
max(casos_marzo)    #> 1500
```

| Familia | Devuelve |
|---|---|
| **Elemento a elemento** (`+`, `*`, `>`) | un vector **del mismo largo** |
| **Agregación** (`sum`, `mean`, `min`, `max`) | **un solo número** |

Tener clara esa distinción es lo que hace que `colSums()` y `rowSums()`, más abajo, no necesiten explicación.

### 3.3.6. Cuando los vectores no miden lo mismo: el reciclado

Solo dos estados mandaron el reporte de abril:

```r
casos_abril <- c(CDMX = 1300, Jalisco = 900)

casos_marzo + casos_abril
#>      CDMX   Jalisco   Yucatan    Puebla NuevoLeon   Chiapas
#>      2500      1750      1730      1570      2800      1820
```

R **no da error**. Pero Yucatán lleva sumados los casos de CDMX, y Puebla los de Jalisco: cuatro de los seis números son inventados. Todo se ve bien —seis estados, nombres correctos, cifras plausibles—, y esa es la trampa: los nombres los pone el vector **largo**, así que la etiqueta confirma un resultado que no existe.

Lo que pasó se llama **reciclado**: R repite el vector corto hasta emparejar al largo.

| | CDMX | Jalisco | Yucatán | Puebla | N. León | Chiapas |
|---|---|---|---|---|---|---|
| `casos_marzo` | 1200 | 850 | 430 | 670 | 1500 | 920 |
| `casos_abril` | 1300 | 900 | *1300* | *900* | *1300* | *900* |
| **suma** | 2500 | 1750 | **1730** | **1570** | **2800** | **1820** |

En cursiva, lo que R repitió sin pedirlo; en negrita, los cuatro resultados falsos. El resultado tiene siempre el largo del vector **más largo**.

El reciclado no es un error en sí: `casos_marzo * 2` es exactamente el mismo mecanismo, con un 2 que se repite seis veces. El problema es no verlo.

```r
c(1, 2, 3) + c(1, 2, 3, 4, 5)
#> [1] 2 4 6 5 7
#> Warning: longitud de objeto mayor no es múltiplo de la longitud de uno menor

c(1, 2, 3) + c(1, 2, 3, 4, 5, 6)
#> [1] 2 4 6 5 7 9
```

**Advertencia**

El aviso salta **solo** si el largo mayor no es múltiplo del menor. La segunda suma no dice ni una palabra: 6 sobre 3, 200 sobre 100 o 12 sobre 6 pasan en silencio, y son justo los casos frecuentes con datos reales. Un aviso tampoco detiene el cálculo: si aparece, la respuesta está mal aunque se vea bien.
Antes de operar dos vectores: `length()` a los dos.

**Ejercicio 3** · letalidad de enero

La letalidad se define como $L = \frac{D}{C} \times 100$, donde $D$ son las defunciones y $C$ el total de casos.
**¿Qué estados tuvieron una letalidad mayor a 3 en el mes de enero?**

```r
casos_enero       <- c(3000, 1500, 1200, 1800, 2500, 2100)
defunciones_enero <- c(100, 60, 25, 30, 80, 55)
names(casos_enero) <- names(defunciones_enero) <- names(casos_marzo)
```

**Respuesta.** Dos líneas: una calcula, otra filtra.

```r
letalidad <- (defunciones_enero / casos_enero) * 100
letalidad
#>      CDMX   Jalisco   Yucatan    Puebla NuevoLeon   Chiapas
#>  3.333333  4.000000  2.083333  1.666667  3.200000  2.619048

letalidad[letalidad > 3]
#>      CDMX   Jalisco NuevoLeon
#>  3.333333  4.000000  3.200000
```

Fíjate en que los nombres **viajan con los datos** a través de la división y del filtro. No hubo que volver a decir qué estado era cada número: por eso valía la pena ponerlos.

**Ejercicio 4** · enero contra marzo

**¿En qué estados la letalidad fue mayor en enero que en marzo?**

**Respuesta.**

```r
letalidad_marzo <- (defunciones_marzo / casos_marzo) * 100
```

```r
letalidad > letalidad_marzo
#>      CDMX    Jalisco    Yucatan     Puebla NuevoLeon    Chiapas
#>     FALSE      FALSE      FALSE      FALSE      FALSE      FALSE

letalidad[letalidad > letalidad_marzo]
#> named numeric(0)
```

> **!** Importante
>
> `named numeric(0)` **no es un error**. Dice cuatro cosas: es un vector, es numérico, tiene nombres y su longitud es cero. En ningún estado se cumplió la condición, y esa es la respuesta correcta.

Un resultado vacío asusta la primera vez. Verlo a propósito, en un caso donde «ninguno» es la respuesta buena, evita leerlo como fallo la próxima vez.

## 3.4. Matrices

Una **matriz** es una estructura bidimensional —filas y columnas, como una tabla— en la que **todos los elementos son del mismo tipo**.

Aparece cuando el vector se queda corto. Por ejemplo, con la composición nutricional de cuatro alimentos por cada 100 g:

| | kcal | Carbohidratos | Proteínas | Grasas |
|---|---|---|---|---|
| Manzana | 52 | 14 | 0.3 | 0.2 |
| Pollo (pechuga) | 165 | 0 | 31 | 3.6 |
| Arroz blanco | 130 | 28 | 2.7 | 0.3 |
| Frijoles | 127 | 23 | 9 | 0.5 |

Se puede hacer con cuatro vectores. Pero entonces la relación entre ellos —que la primera posición de los cuatro es la manzana— deja de estar en los datos y pasa a estar en tu cabeza.

### 3.4.1. R llena las matrices por columnas

```r
matrix(1:9, nrow = 3, ncol = 3)
#>      [,1] [,2] [,3]
#> [1,]    1    4    7
#> [2,]    2    5    8
#> [3,]    3    6    9
```

> ⚠ Advertencia
>
> El 1, 2, 3 quedó en la **primera columna**, no en la primera fila. Es la sorpresa clásica, y si no se detecta aquí se arrastra hasta la matriz de nutrientes, donde ya no es evidente porque los números no están ordenados.

Prueba `matrix(1:12, nrow = 3, ncol = 4)` y busca dónde cae el 4.

Cuando cada vector es **una fila** —un alimento— hay que decirlo:

```
manzana <- c(52, 14, 0.3, 0.2);  pollo  <- c(165, 0, 31, 3.6)
arroz   <- c(130, 28, 2.7, 0.3); frijol <- c(127, 23, 9, 0.5)

nutricion_matrix <- matrix(c(manzana, pollo, arroz, frijol),
                           nrow = 4, byrow = TRUE)
```

Sin `byrow = TRUE` la matriz sale transpuesta y con el sentido físico equivocado: la primera columna sería «manzana» y la primera fila «las calorías de los cuatro».

### 3.4.2. Nombres de filas y de columnas

```
colnames(nutricion_matrix) <- c("Calorias", "Carbohidratos",
                                "Proteinas", "Grasas")
rownames(nutricion_matrix) <- c("Manzana", "Pollo", "Arroz", "Frijoles")
nutricion_matrix
#>          Calorias Carbohidratos Proteinas Grasas
#> Manzana        52            14       0.3    0.2
#> Pollo         165             0      31.0    3.6
#> Arroz         130            28       2.7    0.3
#> Frijoles      127            23       9.0    0.5
```

En vectores era `names()`. En matrices **no**: hay dos dimensiones y hay que especificar cuál. Intentar `names(matriz) <- ...` no falla con un mensaje claro; simplemente no hace lo que esperabas.

### 3.4.3. Añadir filas y columnas

```
huevo    <- c(155, 1.1, 13, 11)
lentejas <- c(116, 20, 9, 0.4)
nutricion_extendida <- rbind(nutricion_matrix, rbind(huevo, lentejas))

fibra <- c(2.4, 0, 0.1, 7.0, 0, 8.0)
sodio <- c(1, 74, 1, 2, 124, 2)
nutricion_final <- cbind(nutricion_extendida, Fibra = fibra, Sodio_mg = sodio)
```

`rbind()` pega **filas** (*row bind*) y `cbind()` pega **columnas** (*column bind*). Las columnas nuevas llevan **seis** valores, uno por cada fila que ya tiene la matriz; si el largo no coincide, R recicla en silencio o falla, según el caso.

Un detalle que se ve en la salida: `rbind(huevo, lentejas)` toma los nombres de las variables como nombres de fila, así que `huevo` y `lentejas` salen en minúscula y desentonan con los cuatro de arriba. Es un buen momento para hablar de consistencia en los nombres.

### 3.4.4. Mirar sin imprimir

```
class(nutricion_matrix)
#> [1] "matrix" "array"

str(nutricion_matrix)
#>  num [1:4, 1:4] 52 165 130 127 14 0 28 23 0.3 31 ...
#>  - attr(*, "dimnames")=List of 2
#>   ..$ : chr [1:4] "Manzana" "Pollo" "Arroz" "Frijoles"
#>   ..$ : chr [1:4] "Calorias" "Carbohidratos" "Proteinas" "Grasas"
```

`str()` da una vista compacta de la **estructura**: dimensiones, tipo y una muestra del contenido. `summary()` da un resumen **estadístico** —mínimo, cuartiles, mediana, media, máximo— y solo funciona si la matriz es numérica.

`str()` es la función que más se usa en la vida real y la que menos se enseña. Con una matriz de 4×4 parece innecesaria; con una de veinte mil filas es lo único que se puede hacer.

> **!** Importante
>
> Regla de la casa: **nunca imprimas un objeto del que no sabes el tamaño.** Primero `str()` o `dim()`.

### 3.4.5. Selección: `matriz[fila, columna]`

```
nutricion_final["Manzana", "Calorias"]   #> 52
nutricion_final["Manzana", ]             # toda la fila
nutricion_final[ , "Calorias"]           # toda la columna
nutricion_final[1, 1]                    # lo mismo, por posición
nutricion_final[c(2:3), ]                # varias filas
nutricion_final[ , c("Proteinas", "Grasas")]
```

**Dejar el hueco vacío significa «todas»**, y la coma nunca se omite: es lo que separa filas de columnas. Olvidarla —`nutricion_final["Manzana"]`— no da un error claro, da un resultado raro.

### 3.4.6. Selección por condición

Funciona igual que en los vectores, en la posición de las filas:

```
high_cal <- nutricion_final[, "Calorias"] > 100
high_cal
#>  Manzana    Pollo    Arroz Frijoles    huevo lentejas
#>    FALSE     TRUE     TRUE     TRUE     TRUE     TRUE

nutricion_final[high_cal, ]   # se queda con las cinco filas TRUE
```

### 3.4.7. La coma no es un adorno

```
nutricion_final[high_cal]     # sin la coma
#>  [1] 165.0 130.0 127.0 155.0 116.0   0.0  28.0  23.0   1.1  20.0  31.0   2.7
#> [13]   9.0  13.0   9.0   3.6   0.3   0.5  11.0   0.4   0.0   0.1   7.0   0.0
#> [25]   8.0  74.0   1.0   2.0 124.0   2.0
```

No da error. Con **un solo** argumento, R deja de tratarla como matriz: la aplana a sus 36 celdas, columna por columna, y **recicla** la condición de 6 elementos seis veces. Se queda con 5 de cada 6 celdas: 30 números sueltos que no significan nada. Es el reciclado de los vectores, ahora en dos dimensiones.

> **!** Importante
>
> Si ves números sueltos donde esperabas filas, te faltó la coma. `[filas, columnas]`: las dos posiciones existen siempre, y dejar una vacía significa «todas».

### 3.4.8. Una sola fila deja de ser matriz

```
huevo_fila <- nutricion_final[nutricion_final[, "Sodio_mg"] > 100, ]
huevo_fila
#>      Calorias Carbohidratos     Proteinas        Grasas         Fibra
#>         155.0           1.1          13.0          11.0           0.0
#>      Sodio_mg
#>         124.0
is.matrix(huevo_fila)
#> [1] FALSE
```

Cuando la condición acierta un solo alimento, el resultado ya no es una matriz sino un **vector**, y el nombre «huevo» desapareció. Si inventas una pregunta que acierte una sola fila, vas a ver algo distinto de lo que esperabas y vas a pensar que te equivocaste. Para conservar la forma existe `drop = FALSE`, con **dos** comas:

```r
nutricion_final[nutricion_final[, "Sodio_mg"] > 100, , drop = FALSE]
#>       Calorias Carbohidratos Proteinas Grasas Fibra Sodio_mg
#> huevo      155           1.1        13     11     0      124
```

### 3.4.9. Un `NA` hace una fila fantasma

La matriz de este capítulo no tiene huecos, pero los datos reales sí. Si falta un dato, la condición sale `NA` —que no es `TRUE` ni `FALSE`— y aparece una fila que no existe:

```r
con_na <- nutricion_final
con_na["Arroz", "Fibra"] <- NA    # un dato que falta

con_na[con_na[, "Fibra"] > 5, c("Fibra", "Sodio_mg")]
#>          Fibra Sodio_mg
#> <NA>        NA       NA
#> Frijoles     7        2
#> lentejas     8        2

con_na[which(con_na[, "Fibra"] > 5), c("Fibra", "Sodio_mg")]
#>          Fibra Sodio_mg
#> Frijoles     7        2
#> lentejas     8        2
```

La fila `<NA>` no existe en los datos. `which()` se queda solo con los `TRUE` de verdad, y es el antídoto.

### 3.4.10. Combinar condiciones

Para responder preguntas con más de un criterio se combinan vectores lógicos con tres operadores:

| A | B | A & B | A \| B | !A |
|---|---|---|---|---|
| TRUE | TRUE | TRUE | TRUE | FALSE |
| TRUE | FALSE | FALSE | TRUE | FALSE |
| FALSE | TRUE | FALSE | TRUE | TRUE |
| FALSE | FALSE | FALSE | FALSE | TRUE |

`&` **(Y)** pide las dos; `|` **(O)**, al menos una; `!` **(NO)** invierte. Mira la primera fila: el `|` de R es **inclusivo**, y `TRUE | TRUE` da `TRUE`. En castellano, «o» se oye muchas veces como «uno u otro, pero no los dos»; en R incluye el caso de que se cumplan las dos.

**¿Qué alimentos tienen más de 10 g de proteína Y menos de 15 g de carbohidratos?**

```r
condicion_A <- nutricion_final[, "Proteinas"]     > 10
condicion_B <- nutricion_final[, "Carbohidratos"] < 15

nutricion_final[condicion_A & condicion_B, ]
#>       Calorias Carbohidratos Proteinas Grasas Fibra Sodio_mg
#> Pollo      165           0.0        31    3.6     0       74
#> huevo      155           1.1        13   11.0     0      124
```

Guardar cada condición en su propia variable no es adorno: permite imprimirlas por separado cuando el resultado no es el esperado. Es la técnica de depuración de este bloque, y sin ella una selección compuesta que devuelve algo raro no se puede diagnosticar.

**¿Qué alimentos son ricos en fibra O bajos en sodio?**

```r
mucha_fibra <- nutricion_final[, "Fibra"]    > 5    # 2 alimentos
poco_sodio  <- nutricion_final[, "Sodio_mg"] < 10   # 4 alimentos

nutricion_final[mucha_fibra | poco_sodio, c("Fibra", "Sodio_mg")]
#>          Fibra Sodio_mg
#> Manzana    2.4        1
#> Arroz      0.1        1
#> Frijoles   7.0        2
#> lentejas   8.0        2
```

Dos más cuatro darían seis, y salieron cuatro: frijoles y lentejas cumplen las dos condiciones y aparecen **una sola vez**. Seleccionar no duplica, y el `|` inclusivo deja entrar a quien cumple las dos. Con `&` habrían salido dos.

**Todos los alimentos menos la manzana:**

```r
es_manzana <- rownames(nutricion_final) == "Manzana"
es_manzana
#> [1]  TRUE FALSE FALSE FALSE FALSE FALSE

nutricion_final[!es_manzana, "Calorias"]
#>    Pollo    Arroz Frijoles    huevo lentejas
#>      165      130      127      155      116
```

`rownames()` devuelve un vector de texto; compararlo con `==` produce el vector lógico que `!` invierte entero. El `!` se aplica a **toda** la condición, no al operador: `!(x > 5)`, nunca `x !> 5`, que no existe.

Con esas tres piezas se contestan preguntas más largas.

**¿Qué alimentos tienen más de 5 g de fibra O menos de 1 g de grasa?**

```
nutricion_final[nutricion_final[, "Fibra"]  > 5 |
                nutricion_final[, "Grasas"] < 1, ]
#>          Calorias Carbohidratos Proteinas Grasas Fibra Sodio_mg
#> Manzana        52            14       0.3    0.2   2.4        1
#> Arroz         130            28       2.7    0.3   0.1        1
#> Frijoles      127            23       9.0    0.5   7.0        2
#> lentejas      116            20       9.0    0.4   8.0        2
```

**¿Cuáles tienen poco sodio y poca grasa, pero no son la manzana?**

```
low_sod  <- nutricion_final[, "Sodio_mg"] < 10
low_gras <- nutricion_final[, "Grasas"]   < 10

nutricion_final[low_sod & low_gras & !es_manzana, ]
#>          Calorias Carbohidratos Proteinas Grasas Fibra Sodio_mg
#> Arroz         130            28       2.7    0.3   0.1        1
#> Frijoles      127            23       9.0    0.5   7.0        2
#> lentejas      116            20       9.0    0.4   8.0        2
```

> **Nota**
>
> Dos detalles para cuando se encadenan más condiciones. **Precedencia**: primero `!`, luego `&`, luego `|`, así que `a | b & c` se lee `a | (b & c)`; con más de dos condiciones, paréntesis. Y **`&&` no es lo mismo que `&`**: `&&` solo admite un valor, y con dos condiciones de seis alimentos da el error `'length = 6' in coercion to 'logical(1)'`. Para seleccionar filas, siempre `&` y `|` sencillos.

### 3.4.11. Resúmenes por dimensión

```
colSums(nutricion_matrix)
#>      Calorias Carbohidratos     Proteinas        Grasas
#>         474.0          65.0          43.0           4.6

colMeans(nutricion_matrix)
#>      Calorias Carbohidratos     Proteinas        Grasas
#>        118.50         16.25         10.75          1.15
```

```
macros <- nutricion_final[, c("Carbohidratos", "Proteinas", "Grasas")]
rowSums(macros)
#>  Manzana    Pollo    Arroz Frijoles    huevo lentejas
#>     14.5     34.6     31.0     32.5     25.1     29.4
```

> **Advertencia**
>
> La suma de la columna **Calorías** existe como número y no significa nada: son kcal de alimentos distintos. `rowSums(macros)` **sí** significa algo —gramos de macronutrientes por alimento— porque las tres columnas comparten unidad.
> R calcula lo que le pidas. No valida el sentido de la operación; eso te toca a ti.

Y cuando se suma una fila **entera**, mezclando unidades:

```
rowSums(nutricion_final)
#>  Manzana    Pollo    Arroz Frijoles    huevo lentejas
#>     69.9    273.6    162.1    168.5    304.1    155.4

which.max(rowSums(macros))
#> Pollo
#>     2
```

El 273.6 del pollo suma kilocalorías, gramos y miligramos de sodio en un mismo número: no significa nada. `rowSums(macros)` es la misma función sobre columnas que sí comparten unidad, y por eso se puede interpretar: el pollo es el alimento con más gramos de macronutrientes. `which.max()` contesta **cuál**, con su nombre y su posición, en vez de **cuánto**.

Con las medias pasa lo mismo: de `colMeans(nutricion_final)` solo tiene sentido reportar las de columnas que comparten unidad **y** contexto. La media de `Sodio_mg` mezcla alimentos muy distintos y dice poco.

## 3.5. Marcos de datos (data frames)

¿Y cuando cada columna es de un tipo distinto? El nombre de un alimento es texto, sus calorías son un número y si es vegetariano es un lógico. Una matriz no puede: exige un solo tipo.

Para eso está el **data frame**, que es:

- **una lista de vectores**: cada columna es un vector;
- de **tipos distintos**: numérico, carácter, lógico, factor, todos juntos;
- del **mismo largo**: todas las columnas con el mismo número de filas.

```
nutricion_df <- data.frame(
  Nombre      = c("Manzana", "Pollo", "Arroz",
                  "Frijoles", "Huevo", "Lentejas"),
  Calorias    = c(52, 165, 130, 127, 155, 116),
  Grupo       = c("Fruta", "Proteína", "Carbohidrato",
                  "Legumbre", "Proteína", "Legumbre"),
  Vegetariano = c(TRUE, FALSE, TRUE, TRUE, FALSE, TRUE)
)
```

Esta es la estructura de casi todo conjunto de datos real: cualquier CSV o archivo de Excel que cargues se convierte en un data frame.

### 3.5.1. Inspeccionarlo

```
str(nutricion_df)
#> 'data.frame':    6 obs. of  4 variables:
#>  $ Nombre     : chr  "Manzana" "Pollo" "Arroz" "Frijoles" ...
#>  $ Calorias   : num  52 165 130 127 155 116
#>  $ Grupo      : chr  "Fruta" "Proteína" "Carbohidrato" "Legumbre" ...
#>  $ Vegetariano: logi  TRUE FALSE TRUE TRUE FALSE TRUE
```

`Grupo` sale como `chr`, texto corriente. Es lo mismo que vas a ver al abrir tu propio CSV. Pero esa columna no es texto de verdad: es una **categoría**, y en el apartado de factores se arregla con una línea.

| Función | Para qué |
|---|---|
| `str()` | estructura y tipos — **la primera siempre** |
| `summary()` | resumen estadístico por columna |
| `head()` · `tail()` | las 6 primeras o últimas filas |
| `dim()` · `nrow()` · `ncol()` | tamaño |
| `names()` · `colnames()` | nombres de columna |

`head(nutricion_df, n = 3)` muestra solo las tres primeras.

### 3.5.2. Seleccionar

```
nutricion_df$Calorias        #> [1]  52 165 130 127 155 116
nutricion_df[["Nombre"]]     # útil si el nombre está guardado en una variable
nutricion_df[, 2]            # por posición

nutricion_df[1, ]                           # primera fila, todas las columnas
nutricion_df[c(1, 2), c("Nombre", "Grupo")]  # filas y columnas
```

`$` es el que se usa el noventa por ciento de las veces. Y con **Tab** después del `$`, RStudio ofrece los nombres de columna: es uno de los trucos que más cambian la experiencia del primer mes.

Por qué funcionan también los dos corchetes, `[[ ]]`, se explica al final del capítulo, en «El data frame es una lista».

### 3.5.3. Filtrar por condición

```
nutricion_df[nutricion_df$Calorias > 120, ]
#>     Nombre Calorias        Grupo Vegetariano
#> 2    Pollo      165     Proteína       FALSE
#> 3    Arroz      130 Carbohidrato        TRUE
#> 4 Frijoles      127     Legumbre        TRUE
#> 5    Huevo      155     Proteína       FALSE

nutricion_df[nutricion_df$Vegetariano == TRUE &
             nutricion_df$Calorias < 120, ]
#>     Nombre Calorias    Grupo Vegetariano
#> 1  Manzana       52    Fruta        TRUE
#> 6 Lentejas      116 Legumbre        TRUE

nutricion_df[!nutricion_df$Vegetariano, ]
#>   Nombre Calorias    Grupo Vegetariano
#> 2  Pollo      165 Proteína       FALSE
#> 5  Huevo      155 Proteína       FALSE
```

Es el **mismo mecanismo** que en vectores y en matrices: una condición que produce `TRUE`/`FALSE`, metida entre corchetes. Tercera vez que aparece en este capítulo, y no son tres reglas: es una.

Los números de fila que sobreviven —2, 3, 4, 5, luego 1, 6— son los del data frame original, no una renumeración.

### 3.5.4. Añadir una columna

```
nutricion_df$Proteinas <- c(0.3, 31, 2.7, 9, 13, 9)
```

Eso es todo: asignarle un vector con `$`. El vector tiene que tener exactamente tantos elementos como filas. Si tiene menos y su largo es divisor del número de filas, R lo **recicla** sin avisar; si no lo es, falla:

```
nutricion_df$Malo <- c(1, 2, 3, 4, 5)   # 5 valores para 6 filas
#> Error: replacement has 5 rows, data has 6
```

Es de los pocos errores de este capítulo que avisan bien. El aviso práctico es «cuenta tus filas».

### 3.5.5. Un data frame de verdad: `cars`

R trae decenas de conjuntos de datos listos para usar, y `data()` los enumera. Sirven para probar código sin tener datos a mano, y son los que vas a encontrar en cualquier tutorial. `cars` tiene 50 frenadas medidas en los años veinte: la velocidad, en millas por hora, y la distancia de frenado, en pies.

```
str(cars)
#> 'data.frame':    50 obs. of  2 variables:
#>  $ speed: num  4 4 7 7 8 9 10 10 10 11 ...
#>  $ dist : num  2 10 4 22 16 10 18 26 34 17 ...

nrow(cars)   #> 50
```

Con cincuenta filas se aplica la regla de la casa: nada de escribir `cars` e imprimirlas todas; primero `str()`, después `head()`.

```
rapidos <- cars$speed > 20   # sum(rapidos) -> 7
cerca   <- cars$dist  < 60   # sum(cerca)   -> 38

cars[rapidos & cerca, ]
#>    speed dist
#> 45    23   54
```

Con 50 filas ya no se puede comprobar a ojo si una selección salió bien. Por eso aquí guardar las condiciones aparte deja de ser cosmético: `sum()` sobre cada una dice cuántas filas la cumplen antes de combinarlas.

| Condición | Frenadas de 50 |
|---|---|
| `rapidos` | 7 |
| `cerca` | 38 |
| `rapidos & cerca` | **1** |
| `rapidos \| cerca` | **44** |
| `!rapidos` | **43** |

Siete más treinta y ocho son cuarenta y cinco, pero el `|` da **44**: la frenada que cumple las dos condiciones se cuenta una sola vez. Y `!rapidos` da 43, que son $50 - 7$: el `!` siempre reparte el total en dos.

> **Nota**
>
> Quedó **una sola fila y sigue siendo un data frame**. En una matriz habría dejado de serlo; es la diferencia que se vio en «Una sola fila deja de ser matriz».

Después de filtrar conviene mirar `nrow()`: si quedan 0 filas, o quedan todas, casi siempre la condición estaba mal.

## 3.6. Factores

En `nutricion_df`, la columna `Grupo` guarda texto. Pero lo que representa no es texto libre: es una **categoría**, y solo puede tomar unos cuantos valores. Para eso existen los factores. *Advanced R* (Wickham, 2019a) los define así:

Un factor es un vector que solo puede contener valores predefinidos. Se usa para guardar datos categóricos. Los factores se construyen sobre un vector de enteros con dos atributos: una clase, `"factor"`, que hace que se comporte distinto de los vectores de enteros normales, y unos niveles (`levels`), que definen el conjunto de valores permitidos.

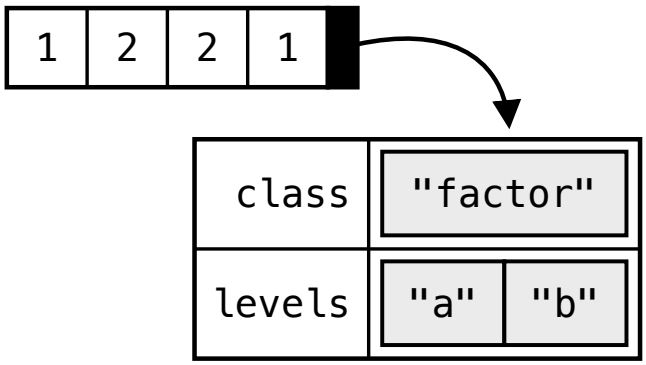



Figura 3.1: El factor `factor(c("a", "b", "b", "a"))` por dentro. Redibujada de Wickham (2019), *Advanced R*, 2.ª ed., §3.4.1, bajo licencia CC BY-NC-SA 4.0.

Las dos primeras frases dicen qué **representa** un factor: una variable categórica. La tercera dice cómo lo **guarda** R: arriba, en la figura, los enteros `1 2 2 1`; abajo, los dos atributos pegados al vector. «Atributo» ya apareció con `names`: información extra que viaja con el vector.

### 3.6.1. Dónde está el factor entre los vectores

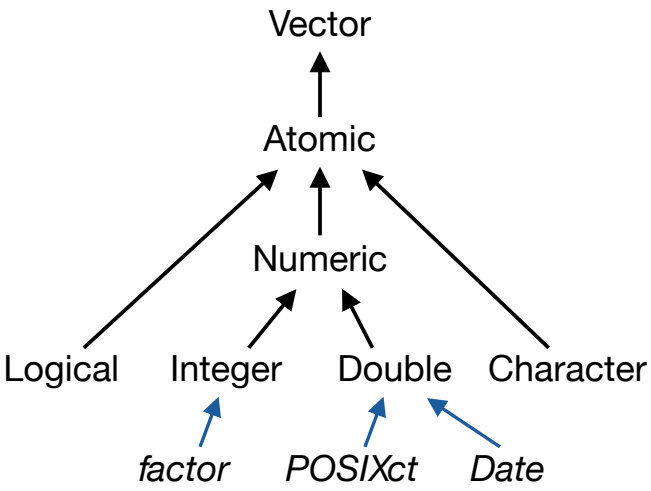



Figura 3.2: Los vectores atómicos y lo que se construye sobre ellos. Redibujada de Wickham (2019), *Advanced R*, 2.ª ed., §3.4, bajo licencia CC BY-NC-SA 4.0.

En negro están los tipos básicos: lógico, entero, doble y carácter; `integer` y `double` son los numéricos. En azul está lo que se construye **encima** de un tipo básico añadiéndole atributos: `factor` sobre

`integer`, y las fechas, `Date` y `POSIXct`, sobre `double`.

Por eso un factor no es una estructura aparte: por dentro es un vector de enteros, y lo que lo vuelve factor son sus atributos. El libro llama a los azules «vectores S3»; para este curso basta con «un tipo básico con atributos».

### 3.6.2. Cuando el texto es en realidad una categoría

```
nutricion_df$Grupo <- factor(nutricion_df$Grupo)
nutricion_df$Grupo
#> [1] Fruta        Proteína     Carbohidrato Legumbre     Proteína
#> [6] Legumbre
#> Levels: Carbohidrato Fruta Legumbre Proteína
```

Ya no hay comillas, y abajo aparecen los **niveles**: las categorías que R encontró, en orden alfabético. Por dentro, R no guarda seis textos. Guarda seis enteros y la lista de niveles una sola vez: si «Proteína» se repite mil veces, guarda mil veces el número 4 y una sola vez la palabra.

> **Nota**
>
> Si encuentras tutoriales viejos: antes de R 4.0, `read.csv()` convertía el texto en factor por su cuenta. Ahora llega como `character`, y el factor lo creas tú cuando lo necesitas.

### 3.6.3. Los datos y el catálogo

```
sev <- c("grave", "leve", "leve", "moderado", "leve")
table(sev)
#> sev
#>    grave     leve moderado
#>        1        3        1

f <- factor(sev, levels = c("leve", "moderado", "grave", "muy grave"))
table(f)
#> f
#>      leve  moderado     grave muy grave
#>         3         1         1         0
```

Con `levels` se declara el **catálogo** de respuestas posibles. Entre las dos tablas cambian dos cosas: el orden pasa a ser el clínico, y aparece «muy grave» con cero. Ese cero es información: en esta muestra no hubo ningún caso muy grave, y el texto no podía decirlo porque nunca supo que esa respuesta existía.

> **!** Importante
>
> El factor **contiene los datos** —`length(f)` es 5, los cinco pacientes— y **además conoce** las respuestas posibles: `nlevels(f)` es 4.

### 3.6.4. Un conjunto cerrado

```
txt <- c("leve", "moderado", "grave")
txt[1] <- "cualquier cosa"   # el texto lo acepta sin quejarse

fac <- factor(c("leve", "moderado", "grave"))
fac[1] <- "cualquier cosa"
#> Warning: invalid factor level, NA generated
fac
#> [1] <NA>     moderado grave
#> Levels: grave leve moderado
```

Lo que no se declaró no existe: el factor lo convierte en `NA`. Es un **aviso, no un error**, así que el código sigue corriendo y el dato se pierde. Pasa al recodificar a mano o al corregir errores de captura. Para añadir una categoría nueva, primero se agrega a los niveles y después se asigna el valor.

### 3.6.5. Contar categorías

```
summary(nutricion_df$Grupo)
#> Carbohidrato        Fruta     Legumbre     Proteína
#>            1            1            2            2
```

> **!** Importante
>
> `summary()` de una matriz numérica daba cuartiles. De un factor da un **conteo**. La misma función se comporta según el tipo del objeto que le des.
> Ese es un rasgo grande del diseño de R, y es la razón por la que `class()` importa tanto.

`table(nutricion_df$Grupo)` da un resultado equivalente.

### 3.6.6. Lo que se ve y lo que se guarda

```
f <- factor(c("Grave", "Leve", "Moderado", "Leve"))

typeof(f)              #> "integer"
class(f)               #> "factor"
```

```r
names(attributes(f))   #> "levels" "class"

str(f)
#>  Factor w/ 3 levels "Grave","Leve",..: 1 2 3 2
```

Tres piezas: los **códigos** enteros, el **catálogo** (`levels`) y la **etiqueta** (`class`), que le dice a R que busque cada código en el catálogo. `str()` las enseña juntas: primero los niveles, al final los códigos `1 2 3 2`.

«Grave» es el 1 porque es el primero del catálogo. El entero no es arbitrario: es la posición en el catálogo, y ahí vive el orden. Y `factor()` ordena el catálogo, no los datos: el primer paciente sigue siendo el primero.

Ahorrar memoria es un motivo secundario: con cien mil valores de cuatro categorías, el factor ocupa la mitad que el texto, no mucho menos.

### 3.6.7. Declarar el orden: `levels` y `relevel()`

```r
x <- c("grave", "leve", "moderado", "leve")

levels(factor(x))
#> [1] "grave"    "leve"     "moderado"
levels(factor(x, levels = c("leve", "moderado", "grave")))
#> [1] "leve"     "moderado" "grave"
levels(relevel(factor(x), ref = "leve"))
#> [1] "leve"     "grave"    "moderado"
```

Sin decir nada, el catálogo sale en orden alfabético. Con `levels` se declara completo. `relevel()` mueve **una sola** categoría al principio y deja el resto como estaba: sirve para elegir la categoría de referencia de un modelo, que se ve al final de este apartado. En los tres casos los datos no se mueven; solo cambia el catálogo.

> **Advertencia**
>
> Un valor que no esté en `levels` se vuelve `NA`, y al crear el factor **ni siquiera avisa**. Antes de declarar, mira qué hay con `unique()`:
>
> ```r
> x2 <- c("grave", "Leve", "moderado", "leve")
> unique(x2)
> #> [1] "grave"    "Leve"     "moderado" "leve"
> factor(x2, levels = c("leve", "moderado", "grave"))
> #> [1] grave    <NA>     moderado leve
> #> Levels: leve moderado grave
> ```
>
> Un «Leve» con mayúscula basta para perder un dato.

### 3.6.8. Categorías que tienen orden

Algunas variables categóricas tienen un orden intrínseco: «leve», «moderado» y «grave» no son intercambiables, ni tampoco los estadios de una clasificación clínica.

```
porciones <- c("Mediana", "Pequeña", "Grande", "Mediana", "Pequeña")

factor_porciones <- factor(porciones,
                           levels = c("Pequeña", "Mediana", "Grande"),
                           ordered = TRUE)
factor_porciones
#> [1] Mediana Pequeña Grande  Mediana Pequeña
#> Levels: Pequeña < Mediana < Grande

factor_porciones[1] > factor_porciones[2]   # Mediana > Pequeña
#> [1] TRUE
```

> ⚠ El orden alfabético es una trampa
>
> Si no escribes `levels`, R los ordena **alfabéticamente**: Grande < Mediana < Pequeña, que es justo al revés.
>
> ```
> mal <- factor(porciones, ordered = TRUE)
> levels(mal)
> #> [1] "Grande"  "Mediana" "Pequeña"
> mal[1] > mal[2]   # Mediana > Pequeña, ¿verdad?
> #> [1] FALSE
> ```
>
> Lo peligroso es que nada avisa: el factor queda ordenado, las comparaciones funcionan y el resultado está invertido. Sobrevive hasta la gráfica final, donde las categorías salen al revés y nadie sabe por qué.
> **Si el factor es ordenado, `levels` se escribe siempre a mano.**

### 3.6.9. `levels` ordena, `ordered` compara

| Argumento | Qué decide | Para qué sirve |
|---|---|---|
| `levels = c(...)` | el **orden** de las categorías | tablas, gráficas y categoría de referencia |
| `ordered = TRUE` | si se pueden **comparar** con `<` y `>` | escalas con jerarquía real |

Son independientes. Sin `ordered = TRUE`, la comparación no se hace:

```r
sin_orden <- factor(porciones, levels = c("Pequeña", "Mediana", "Grande"))
sin_orden[1] > sin_orden[2]
#> [1] NA
#> Warning: '>' no es significativo para factores
```

Sin orden declarado, R se niega a comparar en vez de inventarse una jerarquía. Para **filtrar** basta `==`, que funciona siempre; `ordered` solo hace falta para preguntar «mayor que», y con él funcionan también `min()` y `max()`.

Otro detalle: `sort()` y `table()` respetan el orden de `levels` aunque el factor no sea ordenado. Y cerrado no implica ordenado: fruta, legumbre y proteína forman un conjunto cerrado sin jerarquía.

### 3.6.10. De números a categorías: `cut()`

```r
edad <- c(23, 37, 45, 61, 72, 58)

cut(edad, breaks = c(18, 40, 60, Inf),
    labels = c("18-39", "40-59", "60+"), right = FALSE)
#> [1] 18-39 18-39 40-59 60+   60+   40-59
#> Levels: 18-39 40-59 60+
```

Grupos de edad, categorías de IMC, rangos de presión arterial: muchas variables categóricas de salud nacen de números, y `cut()` da ese paso. `breaks` marca los cortes y `labels` le pone nombre a cada intervalo.

`right = FALSE` incluye el límite **inferior** de cada intervalo: alguien de 40 años cae en «40-59». Sin él, los intervalos se cierran por la derecha y esos 40 años caerían en «18-39». Con `ordered_result = TRUE`, el resultado es un factor ordenado.

> **!** Importante
>
> Los cortes se deciden por criterio clínico, **antes** de mirar los datos. Y aunque los grupos tengan orden, promediar categorías no tiene sentido: las distancias entre niveles no son iguales.

> Ejercicio 5 · una escala clínica
>
> ```r
> severidad <- c("Grave", "Leve", "Moderado", "Leve", "Grave", "Moderado")
> ```
>
> 1. Conviértelo en un factor **ordenado**, de leve a grave.
> 2. Cuenta cuántos hay de cada nivel.
> 3. Comprueba que `factor_sev[1] > factor_sev[2]` da `TRUE`.
> 4. Ahora hazlo **sin** `levels`, manteniendo `ordered = TRUE`, y compara el resultado del punto 3.

**Respuesta.**

```r
factor_sev <- factor(severidad, levels = c("Leve", "Moderado", "Grave"),
                     ordered = TRUE)
table(factor_sev)
#> factor_sev
#>     Leve Moderado    Grave
#>        2        2        2
factor_sev[1] > factor_sev[2]
#> [1] TRUE

sin_levels <- factor(severidad, ordered = TRUE)
levels(sin_levels)
#> [1] "Grave"    "Leve"     "Moderado"
sin_levels[1] > sin_levels[2]
#> [1] FALSE
```

En el punto 4 el orden quedó Grave < Leve < Moderado: la comparación da `FALSE` y nada avisa. Si además quitas `ordered = TRUE`, obtienes `NA` y un aviso; sin orden declarado, R se niega a comparar.

### 3.6.11. La trampa de `as.numeric()`

```r
edad <- factor(c("45", "30", "62", "30"))

as.numeric(edad)                 #> 2 1 3 1       los códigos, no las edades
as.numeric(as.character(edad))   #> 45 30 62 30   lo correcto
```

Es la consecuencia directa de cómo se guarda un factor: por dentro son enteros, y `as.numeric()` devuelve justo eso. Pasa cuando una columna de edades, de dosis o de días de estancia llega como factor y alguien la convierte «para poder promediarla». El resultado es plausible, no avisa, y el análisis queda mal.

> **!** Importante
>
> Para volver a números, **siempre pasando por `as.character()`**.

> **i** Otra trampa: los niveles vacíos
>
> Al filtrar, los niveles no se van:
>
> ```r
> prot <- nutricion_df[nutricion_df$Grupo == "Proteína", ]
> table(prot$Grupo)
> #> Carbohidrato        Fruta     Legumbre     Proteína
> #>            0            0            0            2
> table(droplevels(prot$Grupo))
> #> Proteína
> #>        2
> ```

Quedaron dos filas, pero siguen los cuatro niveles, así que la tabla y la gráfica muestran categorías vacías. `droplevels()` las quita. En una tesis eso aparece en la Tabla 1 y en las cajas del ggplot.

### 3.6.12. Para qué sirven de verdad

- `lm()`, `glm()` y compañía generan **automáticamente** el análisis por grupo cuando una variable es factor.
- `ggplot2` le da **un color, una faceta o una caja** a cada nivel, como se ve en el capítulo de visualización con ggplot2.
- El orden de los niveles decide el orden en que salen las categorías en cualquier tabla o gráfica.

Que una variable sea factor o texto **cambia el resultado**. No es un tecnicismo, y los tres apartados que siguen lo muestran con datos reales.

### 3.6.13. Datos reales: categorías guardadas como números

```
nacimientos <- MASS::birthwt     # los datos birthwt del paquete MASS
str(nacimientos[, c("low", "smoke")])
#> 'data.frame':    189 obs. of  2 variables:
#>  $ low  : int  0 0 0 0 0 0 0 0 0 0 ...
#>  $ smoke: int  0 0 1 1 1 0 0 0 1 1 ...
table(nacimientos$smoke)
#>   0   1
#> 115  74
```

`birthwt` son 189 nacimientos registrados en 1986 en un hospital de Springfield, Massachusetts: un conjunto clásico de epidemiología. `MASS` ya viene instalado con R, y `MASS::birthwt` toma los datos sin cargar el paquete entero. Es preferible a `library(MASS)`: si `MASS` se carga después de dplyr, tapa su función `select()`.

Para R, `low` y `smoke` son enteros. Que el 1 signifique «bajo peso» (menos de 2.5 kg) o «fumó durante el embarazo» solo lo dice el diccionario de datos, `?MASS::birthwt`. Es lo que vas a encontrar en tus propias bases: sexo, tabaquismo o desenlace capturados como 0/1 o 1/2.

```
nacimientos$smoke <- factor(nacimientos$smoke, levels = c(0, 1),
                            labels = c("no fumó", "fumó"))
nacimientos$low   <- factor(nacimientos$low, levels = c(0, 1),
                            labels = c("peso normal", "bajo peso"))
table(nacimientos$smoke)
#> no fumó    fumó
#>     115      74
```

`levels` dice qué códigos existen y en qué orden; `labels` dice cómo se llama cada uno, **en ese mismo orden**. Es el `labels` de `cut()`, ahora dentro de `factor()`. Si se invierten, las madres que fumaron

quedan etiquetadas como «no fumó», y R no avisa. El 0 va primero para que «no fumó» y «peso normal» queden como categoría de referencia, que es lo que tiene sentido clínico.

```
table(MASS::birthwt$smoke, MASS::birthwt$low)   # antes: códigos
#>      0  1
#>   0 86 29
#>   1 44 30

table(nacimientos$smoke, nacimientos$low)       # después: factores
#>           peso normal bajo peso
#>   no fumó          86        29
#>   fumó             44        30
```

Mismos números; solo la segunda tabla se entiende sin abrir el diccionario de datos. Filas y columnas salen, además, en el orden de `levels`. Se lee así: 30 de las 74 madres que fumaron tuvieron un bebé con bajo peso (41 %), contra 29 de las 115 que no fumaron (25 %). Es una asociación en 189 nacimientos, no un análisis ajustado. Los porcentajes por fila salen con `prop.table()`:

```
tabla <- table(nacimientos$smoke, nacimientos$low)
round(prop.table(tabla, margin = 1), 2)
#>           peso normal bajo peso
#>   no fumó        0.75      0.25
#>   fumó           0.59      0.41
```

### 3.6.14. El mismo grupo, dos modelos

`bwt` es el peso al nacer, en gramos, y `race` viene capturada como 1, 2 y 3:

```
# race: 1 = blanca, 2 = negra, 3 = otra
coef(lm(bwt ~ race, data = MASS::birthwt))
#> (Intercept)        race
#>   3230.0900   -154.6132

nacimientos$race <- factor(nacimientos$race, levels = 1:3,
                           labels = c("blanca", "negra", "otra"))
coef(lm(bwt ~ race, data = nacimientos))
#> (Intercept)   racenegra    raceotra
#>   3102.7187   -383.0264   -297.4352
```

Con los códigos, `lm()` ajusta **una sola pendiente**: −155 g «por grupo», como si las distancias entre grupos fueran iguales y «otra» estuviera al doble de distancia de «blanca» que «negra». No significa nada. Como factor, **compara cada grupo con el primero**, «blanca»: sus bebés pesaron 3103 g en promedio; los de madres negras, 383 g menos, y los del grupo «otra», 297 g menos. Eso sí se reporta, como diferencia sin ajustar.

Mismos datos, dos resultados incompatibles, y la diferencia es una línea. Es la primera razón de «Para qué sirven de verdad», ahora con números. (`race` es la clasificación del estudio original y se usa tal como viene en el diccionario de datos.)

Escrito como ecuación se ve qué ajustó `lm()` en cada caso. Con los códigos, `race` es un número y hay una sola pendiente:

$$\text{bwt} = 3230 - 155 \cdot \text{race}$$

Cada paso, de 1 a 2 y de 2 a 3, resta lo mismo: 155 g. Por eso el modelo le da a «negra» unos 2921 g, cuando su promedio real es 2720 g. Como factor, cada grupo tiene su propio coeficiente:

$$\text{bwt} = 3103 - 383 \cdot \text{negra} - 297 \cdot \text{otra}$$

`negra` y `otra` son variables indicadoras: valen 1 si la madre es de ese grupo y 0 si no. En «blanca» las dos valen 0, y por eso 3103 es su promedio. Con los códigos, R traza una recta entre los grupos; como factor, le da a cada grupo su promedio.

### 3.6.15. La categoría de referencia

```
coef(lm(bwt ~ smoke, data = nacimientos))
#> (Intercept)   smokefumó
#>   3055.6957   -283.7767

nacimientos$smoke <- relevel(nacimientos$smoke, ref = "fumó")
coef(lm(bwt ~ smoke, data = nacimientos))
#>  (Intercept) smokeno fumó
#>    2771.9189     283.7767
```

La **primera categoría** es la referencia. En el primer modelo es «no fumó»: sus bebés pesaron 3056 g en promedio, y los de madres que fumaron, 284 g menos. Con `relevel()`, la referencia pasa a «fumó»: 2772 g, y los de madres que no fumaron, 284 g más. Las medias no cambian; cambia contra quién se compara.

Escritas como ecuación, son dos caras del mismo modelo. Con «no fumó» de referencia, y con «fumó»:

$$\text{bwt} = 3056 - 284 \cdot \text{fumó}$$

$$\text{bwt} = 2772 + 284 \cdot \text{no fumó}$$

Al sustituir 0 y 1, las dos dan 3056 g sin fumar y 2772 g fumando. `relevel()` no cambia el ajuste —ni las predicciones ni el $R^2$—: solo cambia desde qué grupo se mide la diferencia.

Se reporta la primera, porque la referencia debe tener sentido clínico: el grupo control, el placebo, la categoría leve, los no expuestos. Si no la declaras, la elige el alfabeto.

Ejercicio 6 · prepara datos reales

```
nacimientos <- MASS::birthwt
# race: 1 = blanca, 2 = negra, 3 = otra
# lwt:  peso de la madre antes del embarazo, en libras
```

1. Corre `coef(lm(lwt ~ race, data = nacimientos))` con `race` tal como viene. ¿Cuántos coeficientes da? Escribe su ecuación.
2. Convierte `race` en factor con esas etiquetas y corre el mismo modelo. ¿Cuántos coeficientes da ahora y cuál es la categoría de referencia?
3. ¿Cuánto pesaban en promedio las madres de cada grupo? ¿El primer modelo cuenta la misma historia?
4. Cambia la referencia a «negra» con `relevel()` y vuelve a correr el modelo. ¿Qué cambió y qué no?

**Respuesta.**

```
coef(lm(lwt ~ race, data = nacimientos))
#> (Intercept)        race
#>  139.963258   -5.495862

nacimientos$race <- factor(nacimientos$race, levels = 1:3,
                           labels = c("blanca", "negra", "otra"))
coef(lm(lwt ~ race, data = nacimientos))
#> (Intercept)   racenegra    raceotra
#>   132.05208    14.75561   -12.03716

nacimientos$race <- relevel(nacimientos$race, ref = "negra")
coef(lm(lwt ~ race, data = nacimientos))
#> (Intercept)  raceblanca    raceotra
#>   146.80769   -14.75561   -26.79277
```

En el punto 1 hay dos coeficientes, una sola pendiente: $\text{lwt} = 140 - 5.5 \cdot \text{race}$. El modelo predice 134.5, 129 y 123.5 libras, siempre bajando de un grupo al siguiente.

En el 2 hay tres coeficientes y la referencia es «blanca», el primer nivel. En el 3 se suma: blanca, 132.1 libras; negra, $132.1 + 14.8 = 146.8$; otra, $132.1 - 12.0 = 120.0$. Son los promedios reales, y cuentan otra historia: las madres negras pesaban casi 15 libras **más** que las blancas, no menos. Una recta no puede contar eso.

En el 4 los promedios no cambian: con «negra» de referencia, 146.8; «blanca», 14.8 menos, y «otra», 26.8 menos. Ojo con la trampa: `relevel()` sin reasignar el resultado a `nacimientos$race` no cambia el modelo. Una libra son unos 0.45 kg.

Ejercicio 7 · un data frame propio

Crea `bebidas_df` con tres bebidas y estas columnas:

| Columna | Valores | Tipo |
|---|---|---|
| `Nombre` | Agua · Jugo de Naranja · Refresco de Cola | carácter |
| `Volumen_ml` | 500 · 250 · 355 | numérico |
| `Tipo` | Bebida sin azúcar · Jugo de fruta · Bebida azucarada | **factor** |

Después:

1. verifica la estructura con `str()`;
2. muestra solo la columna `Volumen_ml`;
3. filtra las bebidas de más de 300 ml;
4. añade una columna lógica `Requiere_Refrigeracion`, `TRUE` solo para el jugo.

La respuesta completa, ejecutada, está en la práctica del tema 02.

## 3.7. Listas

Una **lista** es un contenedor que guarda objetos de **cualquier tipo y cualquier largo**, cada uno en su propio compartimento: vectores, data frames, matrices, incluso otras listas.

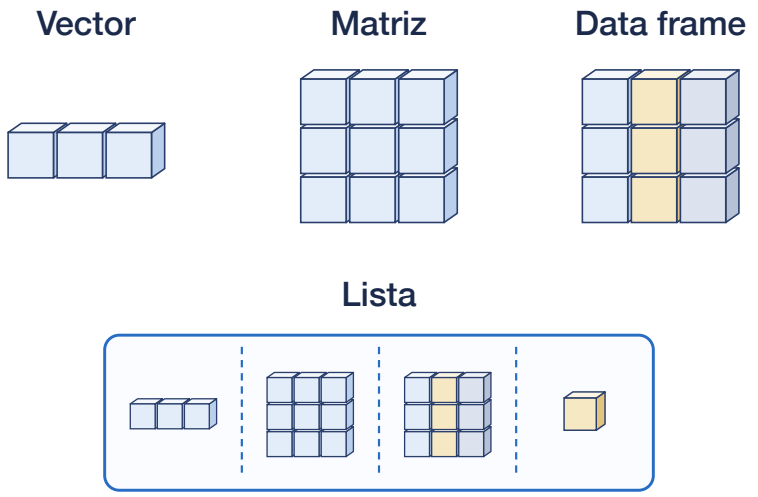



Figura 3.3: Las estructuras del capítulo dibujadas con bloques. En la lista, cada compartimento guarda una estructura distinta. Dibujo propio del curso.

Un vector exige un solo tipo; una matriz, un tipo y dos dimensiones; un data frame, columnas del mismo largo. La lista no exige nada, y por eso es el último escalón.

### 3.7.1. Cuatro cosas que no caben juntas

Un brote en investigación: un nombre, una tabla de casos, quién tuvo contacto con quién y si el brote sigue activo.

```
datos_casos <- data.frame(
  ID_Caso       = c("C01", "C02", "C03"),
  Edad          = c(25, 42, 31),
  Hospitalizado = c(FALSE, TRUE, FALSE)
)
matriz_contactos <- matrix(c(0, 1, 1, 1, 0, 0, 1, 0, 0), nrow = 3)

investigacion_brote <- list(
  Nombre    = "Nuevo brote",     # texto de largo 1
  Datos     = datos_casos,       # un data frame de 3 x 3
  Contactos = matriz_contactos,  # una matriz de 3 x 3
  Activo    = TRUE               # un lógico de largo 1
)
```

Un texto, un data frame, una matriz y un lógico: ni un vector, ni una matriz, ni un data frame pueden guardar las cuatro cosas juntas. Un vector no, porque los tipos son distintos; una matriz tampoco, por lo mismo; un data frame no, porque las piezas no miden lo mismo. En la lista, cada pieza entra en su compartimento con un nombre: `nombre = objeto`.

### 3.7.2. Mirar una lista

```
length(investigacion_brote)
#> [1] 4
names(investigacion_brote)
#> [1] "Nombre"    "Datos"     "Contactos" "Activo"

str(investigacion_brote)
#> List of 4
#>  $ Nombre   : chr "Nuevo brote"
#>  $ Datos    :'data.frame':   3 obs. of  3 variables:
#>   ..$ ID_Caso      : chr [1:3] "C01" "C02" "C03"
#>   ..$ Edad         : num [1:3] 25 42 31
#>   ..$ Hospitalizado: logi [1:3] FALSE TRUE FALSE
#>  $ Contactos: num [1:3, 1:3] 0 1 1 1 0 0 1 0 0
#>  $ Activo   : logi TRUE
```

`length()` cuenta **compartimentos**, no datos: el data frame entero cuenta como uno solo. `str()` vuelve a ser lo primero que hay que teclear: dice cuántos elementos hay, cómo se llaman y de qué tipo es cada uno, y despliega también lo que hay dentro del data frame.

### 3.7.3. Sacar cosas de una lista: la analogía del tren




Figura 3.4: Un tren y sus pasajeros: por qué la lista necesita doble corchete. Analogía de Wickham (2019), *Advanced R*, 2.ª ed., §4.3.1; dibujo propio del curso.

La distinción que más tiempo hace perder al empezar se entiende con una analogía de *Advanced R* (Wickham, 2019a): cada objeto es un **tren**, cada elemento es un **vagón** y lo que va dentro son sus **pasajeros**.

Arriba, el data frame que ya conoces. `datos_casos["Edad"]` desengancha un vagón y devuelve **un tren más corto**: sigue siendo un data frame. Con la coma, `datos_casos[, "Edad"]` baja a **los pasajeros**: el vector `25 42 31`. La coma funciona porque todos los vagones llevan el mismo número de pasajeros: hay filas.

Abajo, la lista. Cada vagón lleva algo distinto, así que no hay filas ni coma. `investigacion_brote["Nombre"]` sigue siendo un tren; para bajar al pasajero hacen falta **dos corchetes**:

```
class(investigacion_brote["Nombre"])     #> "list"        un tren más corto
class(investigacion_brote[["Nombre"]])   #> "character"   el pasajero

investigacion_brote[, "Nombre"]
#> Error in investigacion_brote[, "Nombre"] : número incorreto de dimensiones
```

El error de la coma sale así, «incorreto», con la errata de la traducción de R.

| Operador | Qué devuelve |
|---|---|
| `["Datos"]` | un tren más corto: sigue siendo una `list` |
| `[["Datos"]]` | el pasajero: el objeto que hay dentro |
| `$Datos` | lo mismo que `[["Datos"]]`, con el nombre escrito tal cual |

En el data frame también funcionan `datos_casos[["Edad"]]` y `datos_casos$Edad`; en «El data frame es una lista», más abajo, se ve por qué.

### 3.7.4. Un corchete de menos

```
investigacion_brote[["Datos"]]$Edad    #> 25 42 31
investigacion_brote["Datos"]$Edad      #> NULL

mean(investigacion_brote["Datos"]$Edad)
#> [1] NA
#> Warning: argument is not numeric or logical: returning NA
```

Con un solo corchete sacas un tren de un vagón, y ese tren no tiene ningún vagón llamado `Edad`: `$Edad` busca en el lugar equivocado. R no da error; devuelve `NULL`, y el problema aparece una línea después, cuando `mean()` recibe algo vacío. El aviso sale en inglés aunque R esté en español: ese mensaje no tiene traducción.

> **!** Importante
>
> Si ves un `NULL` donde esperabas datos, la primera pregunta es: ¿cuántos corchetes pusiste?

### 3.7.5. $: el atajo por nombre

```
investigacion_brote$Nombre    #> "Nuevo brote"
investigacion_brote$Nom       #> "Nuevo brote"   basta con el principio
investigacion_brote$Fecha     #> NULL            no existe, y no avisa
```

`$` equivale a `[[ ]]` con el nombre: baja al pasajero. Es el mismo `$` de los data frames.

Que `$Nom` funcione es cómodo en la consola y peligroso en un script: si la lista llega a tener otro elemento que empiece igual, deja de funcionar. En los scripts, nombres completos. Y el `NULL` de un nombre que no existe es el mismo silencio del apartado anterior: antes que el código, revisa la ortografía.

### 3.7.6. Por posición, y con el nombre en una variable

```
class(investigacion_brote[[2]])        #> "data.frame"

campo <- "Datos"
class(investigacion_brote[[campo]])    #> "data.frame"
investigacion_brote$campo              #> NULL   busca un elemento «campo»
```

`[[ ]]` acepta una posición o un nombre guardado en una variable; `$` solo acepta el nombre escrito tal cual. Por posición funciona, pero es frágil: si alguien añade un elemento antes, la posición 2 ya es otra cosa. El caso de la variable aparece en cuanto recorras varias listas, o varias columnas, con el mismo código.

### 3.7.7. Estructuras dentro de la lista

```
investigacion_brote$Datos$Edad[2]                #> 42
investigacion_brote$Contactos[2, 3]              #> 0
investigacion_brote[["Datos"]][["ID_Caso"]][3]   #> "C03"
```

Se lee **de izquierda a derecha**, un paso a la vez: `investigacion_brote$Datos` es el data frame; `$Edad`, su columna; `[2]`, el segundo valor. Cada estructura conserva su propia forma de seleccionar: la matriz de contactos se sigue indexando con `[fila, columna]` aunque viva dentro de una lista.

### 3.7.8. Añadir, cambiar y quitar elementos

```
investigacion_brote$Fecha <- "2026-09-17"   # añadir: un nombre nuevo
length(investigacion_brote)                 #> 5

investigacion_brote$Activo <- FALSE         # cambiar: un nombre que ya existe

investigacion_brote$Activo <- NULL          # quitar
length(investigacion_brote)                 #> 4
names(investigacion_brote)
#> [1] "Nombre"    "Datos"     "Contactos" "Fecha"
```

La misma línea añade o cambia: si el nombre existe, lo sustituye; si no, crea un compartimento nuevo al final. Es la sintaxis con la que se añade una columna a un data frame, pero aquí no hay que respetar ningún largo: el elemento nuevo puede ser cualquier cosa.

> **Advertencia**
>
> Asignar `NULL` **borra** el compartimento, no lo vacía: lo que había ahí se pierde. Si solo querías vaciarlo, guarda otra cosa, por ejemplo `NA`.

### 3.7.9. Cuando cada caso mide distinto

```
diagnosticos <- list(
  A01 = c("HTA", "DM2"),
  A02 = "asma",
  A03 = c("HTA", "ERC", "anemia")
)
lengths(diagnosticos)
#> A01 A02 A03
#>   2   1   3
diagnosticos$A03[2]
#> [1] "ERC"
```

Cada paciente tiene un número distinto de diagnósticos, y un data frame exige columnas del mismo largo; una lista, no. Esa es la razón de fondo para usar listas con datos clínicos: diagnósticos, medicamentos, consultas o resultados de laboratorio que varían de un paciente a otro.

`lengths()`, con s, da el largo de cada elemento; `length()` daría 3, el número de pacientes.

### 3.7.10. `unlist()`: aplanar, y la coerción que vuelve

```
unlist(diagnosticos)
#>     A011     A012      A02     A031     A032     A033
#>    "HTA"    "DM2"   "asma"    "HTA"    "ERC" "anemia"

unlist(list(1, "a", TRUE))
#> [1] "1"    "a"    "TRUE"
```

`unlist()` convierte la lista en **un vector**, y un vector solo admite un tipo: el número y el lógico se volvieron texto. Es la coerción del principio del capítulo, que reaparece al aplanar. Los nombres se numeran solos (A011, A012…) para no repetirse: la pista de qué diagnóstico es de quién queda en el nombre, pero la estructura se perdió.

Ejercicio 8 · el expediente de un paciente

```
folio       <- "A01"
edad        <- 58
diagnosticos <- c("hipertensión", "diabetes tipo 2")
signos      <- data.frame(visita = 1:3, sistolica = c(150, 142, 138))
```

1. Júntalos en una lista `paciente`, cada uno con su nombre, y revísala con `str()`.
2. ¿Cuántos diagnósticos tiene?
3. Saca la presión sistólica de la **última** visita.
4. Añade `alergias` con «penicilina» y después quita `edad`. ¿Cuántos elementos quedan?
5. ¿Qué devuelve `paciente["signos"]$sistolica`? ¿Por qué?

**Respuesta.**

```
paciente <- list(folio = folio, edad = edad,
                 diagnosticos = diagnosticos, signos = signos)
str(paciente)
#> List of 4
#>  $ folio       : chr "A01"
#>  $ edad        : num 58
#>  $ diagnosticos: chr [1:2] "hipertensión" "diabetes tipo 2"
#>  $ signos      :'data.frame':    3 obs. of  2 variables:
#>   ..$ visita   : int [1:3] 1 2 3
#>   ..$ sistolica: num [1:3] 150 142 138
```

```r
length(paciente$diagnosticos)   #> 2
paciente$signos$sistolica[3]    #> 138

paciente$alergias <- "penicilina"
paciente$edad <- NULL
length(paciente)                #> 4

paciente["signos"]$sistolica    #> NULL
```

En el 3 también vale `tail(paciente$signos$sistolica, 1)`, que no depende de saber cuántas visitas hay. En el 4, tras añadir hay cinco elementos y tras quitar quedan cuatro: `folio`, `diagnosticos`, `signos` y `alergias`. En el 5, un solo corchete devuelve un tren de un vagón, y ese tren no tiene ningún `sistolica`: es la trampa del corchete de menos.

### 3.7.11. El data frame es una lista

```r
nacimientos <- MASS::birthwt

typeof(nacimientos)             #> "list"
length(nacimientos)             #> 10   las columnas son sus elementos

class(nacimientos[["bwt"]])     #> "integer"      el pasajero
class(nacimientos["bwt"])       #> "data.frame"   un tren más corto
```

Un data frame es una lista con una regla extra: todos sus elementos son vectores del mismo largo, y por eso se imprime como tabla. Aquí se explica lo que en data frames se usó sin explicar: `$` y `[[ ]]` funcionan porque **cada columna es un compartimento de una lista**. (Se vuelve a cargar `nacimientos` para empezar limpio, sin los factores que se le agregaron más arriba.)

### 3.7.12. Los modelos son listas

```r
nacimientos$smoke <- factor(nacimientos$smoke, levels = c(0, 1),
                            labels = c("no fumó", "fumó"))
modelo <- lm(bwt ~ smoke, data = nacimientos)

typeof(modelo)   #> "list"
length(modelo)   #> 13
names(modelo)
#>  [1] "coefficients"  "residuals"     "effects"       "rank"
#>  [5] "fitted.values" "assign"        "qr"            "df.residual"
#>  [9] "contrasts"     "xlevels"       "call"          "terms"
#> [13] "model"
```

```
modelo$coefficients
#> (Intercept)   smokefumó
#>   3055.6957   -283.7767
summary(modelo)$r.squared
#> [1] 0.03627047
```

Es el modelo de «La categoría de referencia». `lm()` devuelve una lista de 13 compartimentos: los coeficientes, los residuos, los valores ajustados y, en `contrasts`, cómo se codificó el factor. `coef(modelo)` es lo mismo que `modelo$coefficients`.

El $R^2$ no está en `modelo` sino en `summary(modelo)`, que es **otra** lista. Es bajo porque fumar explica poco de la variación del peso al nacer. Todo lo que hagas con un modelo en tu tesis empieza por saber sacar elementos de una lista: saber extraerlos es lo que permite usar el resultado en lugar de solo mirarlo.

Ejercicio 9 · de vectores a listas

```
nacimientos <- MASS::birthwt
# bwt: peso al nacer (g) · smoke: 1 = fumó · low: 1 = bajo peso (< 2500 g)
```

1. **Vector**: guarda `bwt` en `peso` y cuenta cuántos bebés pesaron menos de 2500 g.
2. **Factor**: convierte `smoke` en factor con las etiquetas «no fumó» y «fumó».
3. **Data frame**: guarda en `fumadoras` solo las madres que fumaron, con las columnas `age` y `bwt`. ¿Cuántas son?
4. **Matriz**: haz la tabla de `smoke` contra `low` y saca con `[fila, columna]` cuántas madres que fumaron tuvieron un bebé con bajo peso.
5. **Lista**: junta `peso`, `fumadoras` y `tabla` en una lista `resumen` y saca de ahí la edad de la primera madre que fumó.

**Respuesta.**

```
peso <- nacimientos$bwt
sum(peso < 2500)                    #> 59

nacimientos$smoke <- factor(nacimientos$smoke, levels = c(0, 1),
                            labels = c("no fumó", "fumó"))

fumadoras <- nacimientos[nacimientos$smoke == "fumó", c("age", "bwt")]
nrow(fumadoras)                     #> 74

tabla <- table(nacimientos$smoke, nacimientos$low)
tabla
#>
#>            0  1
#>   no fumó 86 29
#>   fumó    44 30
tabla["fumó", "1"]                  #> 30
```

```r
resumen <- list(peso = peso, fumadoras = fumadoras, tabla = tabla)
resumen$fumadoras$age[1]           #> 20
```

Recorre el capítulo en el orden en que cada paso necesita al anterior. En el 1 son 59: los mismos bebés con bajo peso de la tabla de factores (29 + 30). En el 4 también vale `tabla[2, 2]`, por posición: una tabla de dos variables se selecciona como una matriz. En el 5 también vale `resumen[["fumadoras"]][["age"]][1]`.

Tres tropiezos probables:

- Si te saltas el 2, en el 3 `smoke == "fumó"` compara números con texto: da 0 filas y ningún error.
- En el 4, `low` sigue siendo número: las columnas de la tabla se llaman «0» y «1», y se piden entre comillas.
- En el 5, con un solo corchete, `resumen["fumadoras"]$age` da `NULL`.

## 3.8. La escalera, de un vistazo

| Estructura | Guarda | Se rompe cuando… |
|---|---|---|
| **Vector** | un tipo, una dimensión | necesitas filas **y** columnas |
| **Matriz** | un tipo, dos dimensiones | las columnas son de tipos distintos |
| **Data frame** | tipos distintos, mismo largo | los elementos no miden lo mismo |
| **Lista** | cualquier cosa | — no se rompe, por eso es la última |

Y fuera de la escalera, el **factor**: no es un contenedor, es un vector que sabe a qué categoría pertenece cada dato.

No hay que memorizar estructuras. Hay que reconocer **qué te está pidiendo el problema**.

Y sobre todo, lo que se repitió tres veces:

```r
v[v > 30]                          # vector
m[m[, "Calorias"] > 100, ]         # matriz
df[df$Calorias > 120, ]            # data frame
```

Una sola idea —la condición devuelve un vector lógico, ese vector va entre corchetes, R se queda con lo que valga `TRUE`— en tres sintaxis. La misma reaparece en el capítulo de visualización con ggplot2 para decidir qué se dibuja.

## 3.9. Antes de la próxima sesión

1. Termina el ejercicio 7 en un archivo `.R` guardado.
2. Comprueba que `library(ggplot2)` funciona **sin error**.
3. Corre `str(mpg)` y mira qué tipo tiene cada columna.

El punto 3 no es de relleno: `mpg` es el conjunto de datos con el que se trabaja todo el tema 04, y lo primero que haremos es preguntarnos qué columna se puede mapear a qué.

# 4 Visualización con ggplot2

**Tema 04** · unidad práctica

> **Objetivo**
>
> Construir una gráfica por capas con `ggplot2` —datos, estéticas, geometría, facetas, escalas, etiquetas y tema— y explicar qué hace cada capa; y reconocer cuándo el problema no es la gráfica sino la forma de la tabla.

Toda la sesión construye **una sola gráfica**. Empieza vacía y termina publicable, con una capa más cada vez.

## 4.1. El problema del catálogo de gráficos

La mayoría del software de visualización te ofrece un **catálogo**: barras, pastel, dispersión, líneas. Eliges uno y le das tus datos.

Ese enfoque tiene dos defectos, y los dos son estructurales (Wickham et al., 2016):

- **Siempre se queda corto.** Un programa que ofrece una lista de gráficos predefinidos tendrá inevitablemente menos opciones de las que la gente necesita.
- **No tiene estructura profunda.** Cada «tipo de gráfico» es un caso especial, así que el software es innecesariamente complejo y difícil de extender.

El problema no es que falten tipos. Es que **«tipo de gráfico» no es la unidad correcta**.

### 4.1.1. La otra respuesta: una gramática

Hadley Wickham partió de *The Grammar of Graphics* de Leland Wilkinson (Wilkinson, 2005) para construir un sistema más flexible: en vez de **elegir** un gráfico, lo **compones**.

Un idioma no tiene un catálogo de frases: tiene palabras y reglas para combinarlas. Una gráfica, igual.

> **Importante**
>
> Una gráfica es **una traducción**: de tus variables a las propiedades visuales de unos objetos geométricos.
>
> - **Datos**: tus variables (edad, resultado, cilindrada).
> - **Estéticas**: propiedades visuales (eje x, eje y, color, tamaño, forma).

- **Geometrías**: lo que realmente ves (puntos, líneas, barras).

Esa frase es todo el argumento del capítulo. Lo demás son detalles de sintaxis.

### 4.1.2. R base contra ggplot2

| | Gráficos base | `ggplot2` |
|---|---|---|
| Cómo se construye | dibujando **encima** | **componiendo** capas |
| Modificar lo dibujado | no se puede | se cambia la capa |
| Leyendas | a mano | automáticas |
| El objeto | no existe: es tinta | **es un objeto de R** |

Que la gráfica sea un objeto de R es lo que permite guardarla en una variable, añadirle una capa más adelante y reutilizarla.

Esto no significa que los gráficos base sobren: `hist()`, `boxplot()` y `plot()` resuelven una exploración rápida en una sola línea y siguen siendo lo que uno teclea para «ver qué hay». La diferencia importa cuando la gráfica tiene que crecer.

### 4.1.3. Los ocho componentes

| # | Componente | Responde |
|---|---|---|
| 1 | **data** | ¿qué tabla? |
| 2 | **aes** (*aesthetic mappings*) | ¿qué variable a qué propiedad visual? |
| 3 | **geom** (geometrías) | ¿qué objeto se dibuja? |
| 4 | **stat** (estadísticas) | ¿qué se calcula antes de dibujar? |
| 5 | **scale** (escalas) | ¿cómo se traduce el dato al valor visual? |
| 6 | **coord** (coordenadas) | ¿en qué sistema de coordenadas? |
| 7 | **facet** (facetas) | ¿se parte en paneles? |
| 8 | **theme** (tema) | ¿qué aspecto tiene lo que no son datos? |

En este capítulo se usan el 1, 2, 3, 5, 7 y 8. El `stat` aparece de forma implícita y `coord` casi no hace falta al empezar.

No hay que memorizar los ocho. Lo que hay que reconocer es que **cada línea de código que escribas cae en una de estas casillas** — y esa habilidad se transfiere a cualquier otra herramienta de visualización por capas.

## 4.2. Datos ordenados: la condición de entrada

`ggplot2` forma parte de **tidyverse**, una colección de paquetes que comparten una filosofía, una gramática y unas estructuras de datos comunes. La estructura que comparten se llama **datos ordenados** (*tidy data*) y tiene tres reglas (Wickham, 2014):

1. Cada **variable** forma una **columna**.
2. Cada **observación** forma una **fila**.
3. Cada **valor** está en **su propia celda**.

### 4.2.1. Por qué le importa a ggplot2

Porque `aes()` **mapea columnas**. Si la variable no es una columna, no hay nada que mapear.

«Colorea por tipo de tracción» solo funciona si existe **una columna** que contenga el tipo de tracción. Si en vez de eso tuvieras tres columnas —`traccion_4`, `traccion_f`, `traccion_r`— no hay una variable que mapear: hay tres columnas que en realidad son valores de una misma variable.

> **!** Importante
>
> Si la gráfica no sale y el código parece correcto, la causa suele estar **antes del `ggplot()`**: la tabla no tiene la forma que hace falta.
> Es la diapositiva que más frustración ahorra del curso. La mayoría de los «no me sale la gráfica» son problemas de forma de tabla, no de sintaxis.

Reordenar tablas es tema de `tidyr` (`pivot_longer()`, `pivot_wider()`) y queda fuera de este curso. Basta con saber **reconocer** el problema.

Es la misma lección del Capítulo 3 vista desde el otro lado: elegir bien la estructura no es pulcritud, es lo que permite el paso siguiente.

## 4.3. Los datos: `mpg`

```
library(ggplot2)
str(mpg)
```

`mpg` viene dentro de `ggplot2`, así que no hay que descargar nada. Son **234 observaciones y 11 variables** sobre consumo de combustible de automóviles. Las que se usan aquí:

| Columna | Qué es | Tipo |
|---|---|---|
| `displ` | cilindrada, en litros | numérica |
| `hwy` | rendimiento en carretera, millas por galón | numérica |
| `cty` | rendimiento en ciudad, millas por galón | numérica |
| `drv` | tracción: `4`, `f` (delantera), `r` (trasera) | carácter |
| `class` | tipo de vehículo: compacto, SUV… | carácter |

## 4.4. La gráfica mínima

```
ggplot(data = mpg) +
  geom_point(aes(x = displ, y = hwy))
```

Tres piezas y ya hay gráfica:

- `ggplot(data = mpg)` — **qué tabla**;
- `aes(x = displ, y = hwy)` — **qué variable a qué eje**;
- `geom_point()` — **qué se dibuja**.

Datos + estéticas + geometría = una gráfica completa. Todo lo que sigue en este capítulo es refinamiento.

La relación que aparece es **negativa**: a más cilindrada, menos millas por galón.

### 4.4.1. El + no es casualidad

```
ggplot(data = mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  geom_smooth(aes(x = displ, y = hwy))
```

Cada `+` **añade una capa** sobre la anterior, y el orden importa: lo último se dibuja encima.

`geom_smooth()` añade una línea de tendencia, y es un buen ejemplo de una capa que **calcula algo antes de dibujar** — ahí está el `stat` de la lista de ocho.

Conviene notar por qué es `+` y no el `%>%` o el `|>` del resto de tidyverse: no se está pasando un resultado a la función siguiente, se está **sumando** una capa a un objeto.

### 4.4.2. El mapeo por defecto

Repetir el `aes()` en cada capa no hace falta:

```
ggplot(data = mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth()
```

- El `aes()` que va en `ggplot()` es el **mapeo por defecto** de todas las capas.
- El `aes()` que va dentro de un `geom` **solo aplica a esa capa**.

Regla práctica: lo que comparten todas las capas va arriba; lo que es propio de una capa va en esa capa. No es una convención de estilo, cambia el resultado.

## 4.5. Estéticas

Una **estética** es una propiedad visual que puede ser controlada por una variable de tus datos. Es el «cómo» se va a representar la información.

| Estética | Qué controla | Sirve para |
|---|---|---|
| `x`, `y` | posición | cualquier variable |
| `colour` | color del trazo | categorías o números |
| `fill` | color de relleno | barras, cajas, áreas |
| `size` | tamaño | números |
| `shape` | forma del punto | categorías, **pocas** |
| `alpha` | transparencia | nubes densas |

`shape` admite unas seis formas distinguibles a simple vista; con más categorías, deja de leerse.

### 4.5.1. Mapear no es fijar

```
# Mapea: el color DEPENDE de una variable
ggplot(mpg, aes(x = displ, y = hwy, colour = drv)) + geom_point()

# Fija: el color es azul, sin más
ggplot(mpg, aes(x = displ, y = hwy)) + geom_point(colour = "blue")
```

> **!** Importante
>
> Dentro de `aes()` van **variables**. Fuera de `aes()` van **valores**.

Es la regla que más errores evita del capítulo, y el error que produce saltársela es característico:

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(colour = "blue"))
```

> **⚠** Advertencia
>
> Esto **no pinta de azul**. Crea una variable con un solo valor —el texto `"blue"`—, la mapea al color y dibuja una leyenda que dice «blue».

Merece la pena correrlo a propósito una vez: la leyenda absurda lo explica mejor que cualquier párrafo.

## 4.6. Geometrías

Un **geom** es el objeto geométrico que se usa para representar los datos. Se añaden al gráfico con el signo `+`.

| Geom | Dibuja | Necesita |
|---|---|---|
| `geom_point()` | puntos | `x`, `y` |
| `geom_line()` | líneas | `x`, `y` |
| `geom_bar()` | barras — **cuenta filas** | `x` |
| `geom_col()` | barras — **usa el valor que le das** | `x`, `y` |
| `geom_histogram()` | histograma | `x` |
| `geom_boxplot()` | cajas y bigotes | `x` categórica, `y` |
| `geom_smooth()` | línea de tendencia | `x`, `y` |

> **Advertencia**
>
> La confusión clásica de las barras: si tu tabla ya trae la columna con el total, va `geom_col()`; si quieres que R cuente cuántas filas hay de cada categoría, va `geom_bar()`.

El catálogo completo, con figuras de cada uno, está en el capítulo de geoms individuales del libro de `ggplot2` (Wickham et al., 2016).

> **Ejercicio 1 · tu primera gráfica**
>
> 1. Haz una dispersión de `cty` (eje x) contra `hwy` (eje y).
> 2. Colorea los puntos según `class`.
> 3. Añade una línea de tendencia.
> 4. ¿Cómo describirías la relación entre `cty` y `hwy`? ¿Te sorprende?
>
> **Respuesta al punto 4.** Es fuertemente positiva y casi lineal: las dos variables miden lo mismo —consumo— en condiciones distintas, y un coche eficiente lo es en ciudad y en carretera. Que una gráfica confirme lo obvio también es un resultado: sirve para validar que estás leyendo bien los ejes.

## 4.7. Facetas

Un **facet** es una forma de tomar un solo gráfico y dividirlo en múltiples subgráficos, uno por categoría.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_wrap(~ drv)
```

La `~` se lee «en función de». Salen tres paneles —`4`, `f` y `r`— y, lo que importa, **con los mismos ejes**. Tres gráficas separadas con ejes distintos engañan; tres paneles con el mismo eje, no. Esa es la razón de existir de las facetas.

### 4.7.1. Color o faceta: no es lo mismo

| | Colorear por `drv` | Facetar por `drv` |
|---|---|---|
| Los grupos están | superpuestos | separados |
| Comparar **entre** grupos | fácil | cuesta más |
| Ver **la forma** de cada grupo | cuesta si hay solapamiento | fácil |
| Con muchas categorías | ilegible | sigue funcionando |

No hay una que sea mejor. Hay una que responde **tu** pregunta: si preguntas «¿cuál grupo rinde más?», colorea; si preguntas «¿la relación tiene la misma forma en los tres?», facetea.

### 4.7.2. Dos variables a la vez

```
facet_wrap(~ class)     # una variable: paneles envueltos en una rejilla
facet_grid(drv ~ cyl)   # dos variables: filas x columnas
```

> **Advertencia**
>
> `facet_grid()` dibuja **todas** las combinaciones, también las que no tienen datos. Salen paneles vacíos — y eso es información: esa combinación no existe en el conjunto.

Los paneles vacíos molestan a la vista y suelen ser lo más interesante de la figura.

## 4.8. Escalas

Una **escala** es la regla que convierte un valor del dato en el valor visual que ves. Cada estética tiene la suya, y el nombre sigue un patrón:

```
scale_<aes>_<tipo>()
```

```
scale_colour_manual(values = c("red", "blue", "green"))
scale_y_continuous(limits = c(0, 50))
scale_x_log10()
```

> **Importante**
>
> **Siempre hay una escala.** Si no la escribes, `ggplot2` pone una por defecto. Escribirla es **sustituir** la predeterminada, no añadir algo nuevo.

Que la escala siempre exista es lo que hay que entender: los ejes, los colores y la leyenda no aparecen «solos», los pone una capa que no escribiste.

> **Ejercicio 2 · escalas**
>
> Sobre tu última gráfica: colorea los tres tipos de tracción de `"red"`, `"blue"` y `"green"`, y haz que el eje **y** vaya de 0 a 50.

```
ggplot(mpg, aes(x = displ, y = hwy, colour = drv)) +
  geom_point() +
  scale_colour_manual(values = c("red", "blue", "green")) +
  scale_y_continuous(limits = c(0, 50))
```

Dos advertencias sobre ese ejercicio:

- En inglés británico es `colour` y en americano `color`. **`ggplot2` acepta los dos**, pero conviene no mezclarlos dentro del mismo archivo.
- Rojo, azul y verde a la vez es **una mala elección de colores**: el rojo y el verde no se distinguen para una parte de cualquier grupo. El ejercicio es de sintaxis; una paleta de verdad se elige con `scale_colour_viridis_d()` o con `RColorBrewer`.

## 4.9. Etiquetas

Los *labels* son todo el texto descriptivo que le añades a una gráfica para darle contexto: título, subtítulo, etiquetas de los ejes, título de las leyendas y nota al pie.

```
labs(
  title   = "A mayor cilindrada, menor rendimiento",
  x       = "Cilindrada (litros)",
  y       = "Millas por galón en carretera",
  colour  = "Tracción",
  caption = "Datos: conjunto mpg del paquete ggplot2"
)
```

> **Importante**
>
> `displ` y `hwy` son nombres de columna, no etiquetas de eje. Una gráfica que sale de tu archivo de trabajo **siempre** lleva `labs()`.

Dos detalles que separan una figura de trabajo de una de publicación:

- El título que **afirma algo** —«A mayor cilindrada, menor rendimiento»— en vez de describir —«Cilindrada contra rendimiento»—.
- El argumento se llama igual que la estética: `colour = "Tracción"` cambia el título de la **leyenda** de color.

## 4.10. Tema

El **tema** es la capa que ajusta la estética general: fuentes, colores de fondo, líneas de la cuadrícula y posición de las leyendas.

```
theme_bw()        theme_minimal()
theme_classic()   theme_light()
theme_gray()      theme_dark()
```

`theme_gray()` es el predeterminado —el del fondo gris—. `theme_minimal()` y `theme_bw()` son los que mejor se ven proyectados y dentro de un documento.

> **Tema o escala**
>
> El tema **no** controla los colores de los datos: eso son las escalas.
> La pregunta que separa las dos: ¿el color representa un dato? Entonces es escala. ¿Es decorado? Entonces es tema.

## 4.11. La gráfica completa

```
ggplot(mpg, aes(x = displ, y = hwy, colour = drv)) +   # data + aes
  geom_point(alpha = 0.7) +                            # geom
  geom_smooth(se = FALSE) +                            # geom + stat
  facet_wrap(~ drv) +                                  # facet
  scale_y_continuous(limits = c(0, 50)) +              # scale
  labs(title = "A mayor cilindrada, menor rendimiento",
       x = "Cilindrada (litros)", colour = "Tracción") +
  theme_minimal()                                      # theme
```

Ocho líneas, y el comentario de cada una dice a qué casilla de la gramática corresponde. Ese es el cierre del argumento que abrió el capítulo.

Un último detalle: `alpha = 0.7` va **fuera** de `aes()`, porque es un valor y no una variable (Sección 4.5.1).

## 4.12. Lo que te llevas

- Una gráfica es **una traducción** de variables a propiedades visuales.
- **Datos + estéticas + geometría** ya es una gráfica; lo demás es refinar.
- Dentro de `aes()` van **variables**; fuera, **valores**.
- Colorear y facetear responden **preguntas distintas**.
- Si la gráfica no sale, sospecha de **la forma de la tabla** antes que del código.

## 4.13. Lo que te llevas del curso

| Sesión | La idea que queda |
|---|---|
| **01** | R no vale porque calcule: vale porque **deja escrito cómo calculó** |
| **02** | La estructura no es pulcritud: **es lo que permite el paso siguiente** |
| **03** | No elijas un gráfico de un catálogo: **compón el que responde tu pregunta** |

Las tres son la misma idea vista tres veces: **el procedimiento importa tanto como el resultado.**

## 4.14. A dónde seguir

| Si quieres… | Ve a |
|---|---|
| profundizar en `ggplot2` | *ggplot2: Elegant Graphics for Data Analysis* (Wickham et al., 2016) — https://ggplot2-book.org |
| manipular tablas | *R for Data Science* (Wickham et al., 2023) — https://r4ds.hadley.nz |
| entender R por dentro | *Advanced R* (Wickham, 2019b) — https://adv-r.hadley.nz |
| una hoja de referencia | las *cheatsheets* de Posit — https://posit.co/resources/cheatsheets/ |
| practicar sin instalar nada | el paquete `swirl`, dentro de R |

Los tres libros son de acceso libre y están completos en línea.

# Parte I

# Anexos

# Anexo · Posit Cloud

En este curso **no instalas nada**. Trabajamos en Posit Cloud, que es RStudio corriendo dentro del navegador.

## Crear tu cuenta

1. Entra a https://posit.cloud.
2. Elige **Sign Up** y el plan **Cloud Free**.
3. Puedes registrarte con tu correo, o con tu cuenta de Google o de GitHub.
4. Confirma el correo de verificación.

Con eso ya tienes RStudio. No hay descarga ni instalación.

## Qué vas a ver

La misma ventana de RStudio que usarías en tu computadora:

- **Consola**, abajo a la izquierda: donde escribes y R responde.
- **Editor**, arriba a la izquierda: donde guardas tu código.
- **Environment**, arriba a la derecha: los objetos que has creado.
- **Files / Plots / Packages**, abajo a la derecha.

No es una versión reducida ni un simulador: es RStudio. Lo que aprendas aquí funciona igual el día que instales R en tu máquina.

## Por qué en la nube y no en tu computadora

Instalar R y RStudio en cada computadora convierte la primera clase en soporte técnico: versiones distintas, permisos de administrador, rutas con acentos, antivirus. Se va la sesión y nadie aprendió R.

En la nube trabajamos sobre el mismo R, con los mismos paquetes. Si algo falla, falla igual para todo el grupo y se resuelve una vez.

## Límite del plan gratuito

El plan **Cloud Free** incluye un número limitado de horas de cómputo al mes —alrededor de 15— y un tope de proyectos.

Para las sesiones del curso alcanza, pero conviene saber dos cosas:

- **Cierra el proyecto cuando termines.** Un proyecto abierto sigue consumiendo horas aunque no estés escribiendo. Es la causa más común de quedarse sin horas a media semana.
- Puedes revisar tu consumo en la sección de tu cuenta.

> **Nota**
>
> Las condiciones del plan gratuito las fija Posit y cambian de vez en cuando. Verifica el límite vigente en https://posit.cloud/plans.

## Si prefieres instalarlo en tu computadora

Se puede, y al final del curso lo vemos con calma. El orden importa: primero R, después RStudio, porque RStudio necesita que R ya exista para encontrarlo.

- R — https://cran.r-project.org
- RStudio Desktop — https://posit.co/download/rstudio-desktop/

No es requisito del curso. Si decides hacerlo, hazlo **además** de Posit Cloud, no en lugar de: las sesiones se dan sobre el entorno de la nube.

# Anexo · Glosario

Términos técnicos que aparecen en el curso, definidos en el punto donde se usan por primera vez y recogidos aquí para consulta rápida.

**Objeto.** Un nombre al que le asignas un valor para poder reutilizarlo.

**Función.** Una instrucción que recibe algo, hace una operación y devuelve un resultado.

**Vector.** Una secuencia de valores del mismo tipo.

**Data frame.** Una tabla: columnas con nombre, cada una del mismo largo.

**Reproducibilidad.** Que otra persona, con el mismo código y los mismos datos, obtenga el mismo resultado.

**Repositorio.** La carpeta del proyecto con su historial de cambios versionado.

# 5 Anexo · Errores frecuentes

Este anexo está ordenado por **lo que ves en la consola**, no por tema. Cuando algo falla, busca aquí el mensaje.

Casi todos estos errores aparecen a lo largo del curso y ninguno es señal de que lo estés haciendo mal: son la forma en que R avisa, y aprender a leerlos es parte del curso.

## 5.1. Errores que R sí reporta

### 5.1.1. `could not find function "..."`

```
Error in ggplot(...) : could not find function "ggplot"
```

Falta `library(ggplot2)` **en esta sesión**. Instalar y cargar son dos cosas distintas (Sección 2.4.1).

También puede ser una errata: R distingue mayúsculas de minúsculas, así que `Ggplot()` no existe. **Tab** completa el nombre y de paso confirma que existe.

### 5.1.2. `objeto 'x' no encontrado`

```
Error: objeto 'casos' no encontrado
```

Tres causas, en orden de frecuencia:

1. **Todavía no lo creaste.** Suele pasar al correr un script desde la mitad.
2. **Lo escribiste distinto.** `casos` no es `Casos`, y `aun` no es `aún`.
3. **Se te olvidaron las comillas** alrededor de un texto: `nombre <- Luis` hace que R busque un objeto llamado `Luis`.

### 5.1.3. `argumento no-numérico para operador binario`

```
Error in y + num : argumento no-numérico para operador binario
```

Estás sumando un número con un texto. Comprueba con `class()` y convierte con `as.numeric()`. Se ve en el tema de tipos y estructuras de datos.

En datos reales, esto casi siempre significa que una columna que debía ser numérica se leyó como texto porque traía un espacio, un guion o una coma decimal.

### 5.1.4. `subíndice fuera de los límites`

```
Error in x[[10]] : subíndice fuera de los límites
```

Pediste el elemento 10 de algo que tiene menos. Comprueba el tamaño con `length()`, `dim()` o `str()` antes de indexar.

Recuerda que **en R los índices empiezan en 1**, no en 0.

### 5.1.5. `no se puede abrir la conexión / cannot open file`

R no encuentra el archivo. Comprueba dónde está buscando:

```
getwd()             # directorio de trabajo actual
list.files()        # qué hay ahí
```

La solución duradera no es `setwd()`: es trabajar dentro de un **proyecto de RStudio** (`.Rproj`), que fija el directorio de trabajo en la carpeta del proyecto.

## 5.2. Avisos que no son errores, y son peores

Un error para la ejecución. Un aviso la deja seguir con datos equivocados.

### 5.2.1. `NAs introducidos por coerción`

```
as.numeric("hola")
#> [1] NA
#> Warning message: NAs introducidos por coerción
```

Convertiste a número algo que no lo era, y ese valor ahora es `NA`. Si conviertes una columna entera, cuenta cuántos `NA` aparecieron:

```
sum(is.na(mi_columna))
```

Si el número no es cero, ve a mirar esas filas antes de seguir.

### 5.2.2. Coerción silenciosa en un vector

```
c(1, 2, "tres")
#> [1] "1"    "2"    "tres"
```

**Sin ningún aviso.** Tus números se volvieron texto y el error va a aparecer mucho más abajo, cuando un `sum()` falle.

Regla: si una operación numérica falla sin motivo aparente, `class()` al vector antes que nada.

### 5.2.3. `longitud del objeto mayor no es múltiplo de la longitud del menor`

R estaba **reciclando** un vector corto para completar uno largo, y no le salió exacto. El reciclado en sí no avisa cuando el largo es divisor, así que este mensaje es la punta del problema.

Cuenta tus filas antes de asignar una columna nueva.

### 5.2.4. Un factor al que le asignas un nivel que no existe

```
factor_dieta[6] <- "Keto"
#> Warning message: invalid factor level, NA generated
```

Primero se declara el nivel, después se asigna el valor. Los factores se ven en el tema de tipos y estructuras de datos.

## 5.3. Resultados que parecen errores y no lo son

### 5.3.1. `named numeric(0)`, `character(0)`, `integer(0)`

Un vector **vacío**. Ninguna posición cumplió la condición. Es una respuesta válida y a veces es la respuesta correcta.

### 5.3.2. `NULL`

El elemento no existe. En una lista, `mi_lista$Inexistente` devuelve `NULL` en vez de dar error, lo cual es cómodo y también una fuente de confusión.

### 5.3.3. `[1]` al principio de cada línea

No es parte del resultado: es **la posición del primer elemento** que R está imprimiendo en esa línea. Con vectores largos verás `[1]`, `[18]`, `[35]`…

### 5.3.4. Una gráfica que no aparece

Dentro de un script o una función, `ggplot()` construye el objeto pero no lo dibuja si no lo imprimes. En la consola se dibuja solo; dentro de un `for` o de una función, hay que envolverlo en `print()`.

## 5.4. Confusiones de concepto

| Síntoma | Causa real |
|---|---|
| «Instalé el paquete y sigue sin funcionar» | falta `library()` en esta sesión |
| «Le puse `names()` a la matriz y no pasó nada» | en matrices son `rownames()` y `colnames()` |
| «Los números salieron en la columna equivocada» | `matrix()` llena por columnas; falta `byrow = TRUE` |
| «Las categorías salen al revés en la gráfica» | factor ordenado sin `levels` escrito a mano |

«`geom_point(aes(colour = "blue"))` no pinta de azul» | dentro de `aes()` van variables, fuera van valores |

«Mis barras cuentan en vez de usar mi total» | `geom_bar()` cuenta; querías `geom_col()` |

«Perdí todo el trabajo al cerrar RStudio» | estaba en la consola, no en un script guardado |

## 5.5. Cómo pedir ayuda de forma que se pueda contestar

Cuando escribas —al canal del curso o por correo— incluye estas cuatro cosas:

1. **Qué querías hacer**, en una frase.
2. **El código exacto** que corriste, copiado y pegado, no descrito.
3. **El mensaje completo**, copiado y pegado, no una captura recortada.
4. La salida de `sessionInfo()`, que dice tu versión de R y qué paquetes tienes cargados.

Con eso, casi cualquier problema se resuelve en una respuesta. Sin eso, hacen falta tres.

# Referencias


R Core Team. (2024). *An Introduction to R*. R Foundation for Statistical Computing. https://cran.r-project.org/doc/manuals/r-release/R-intro.html

Wickham, H. (2014). Tidy Data. *Journal of Statistical Software*, *59*(10), 1-23. https://doi.org/10.18637/jss.v059.i10

Wickham, H. (2019a). *Advanced R* (2.ª ed.). Chapman; Hall/CRC. https://adv-r.hadley.nz/

Wickham, H. (2019b). *Advanced R* (2.ª ed.). Chapman; Hall/CRC. https://adv-r.hadley.nz

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2.ª ed.). O'Reilly Media. https://r4ds.hadley.nz

Wickham, H., Navarro, D., & Pedersen, T. L. (2016). *ggplot2: Elegant Graphics for Data Analysis* (3.ª ed.). Springer. https://ggplot2-book.org

Wilkinson, L. (2005). *The Grammar of Graphics* (2.ª ed.). Springer. https://doi.org/10.1007/0-387-28695-0